百科荣创
BECSTRONG

EXPERIMENT GUIDE BOOK | 实验指导书

百科荣创（北京）科技发展有限公司
机器视觉检测应用开发平台
Linux 系统基础课程资源

前 言

百科荣创成立于 2003 年，是国内领先的新一代信息技术产业高素质工程技术与技术技能人才培养解决方案提供商，设有北京总部、济南分部（产教融合服务中心、企业公共实习实践基地）、深圳分部（研发制造中心）、成都分部（筹），国家级高新技术企业与双软认证企业，国内电子、通信、嵌入式、物联网、人工智能等电子信息领域教育头部企业，面向全国高校、职业院校提供专业共建、人才联合培养、实验实训装备、课程开发、教材联合开发、学生实习实践与就业等产品与产教融合服务。公司自成立以来，坚持秉承“产品就是人品！做企业就是做人！”的经营理念和“来源于教育，回报于教育”的企业精神，汇聚了一大批电子、通信、嵌入式、物联网、人工智能方向的优秀人才，以培养智能化时代电子信息大类复合型技术技能人才为宗旨，致力于为客户提供最满意的产品和服务，力争成为中国工程教育与职业教育领域的产教融合型领军企业。

十多年来，公司产品行销全国各省、市、自治区、成为众“双一流”高校以及职业院校采购同类产品的首选，公司产品主要涉及职教工程实训和高教实验研究两大领域，涵盖了电子、通信、物联网、移动互联与智能控制、机器人及人工智能等专业学科方向系列实验实训教学及创新设备，用户已遍及清华大学、北京大学、北京理工大学、北京科技大学、天津大学、重庆大学、东北大学、四川大学、深圳职业技术学院、天津电子信息学院、上海电子信息职业技术学院、北京电子科技职业学院等全国各大知名院校。

为了给您提供更加优质的服务，公司设有 400 开头的免费电话，在使用过程中，如果您有任何问题需要咨询或者需要服务，请与本公司相关部门联系，我们定当竭诚为您服务。服务电话：400-067-7899。

欢迎您在使用的过程中为我们提出宝贵的意见和建议。

目 录

第 1 章 Linux 系统基础知识 1
1.1 Linux 系统 1
1.2 Linux 常用基础命令（一） 6
1.3 Linux 常用基础命令（二） 12
1.4 Linux 权限 27
1.5 shell 环境 37
1.6 文本编辑器 41
1.7 存储媒介 45
1.8 网络相关 49
1.9 文件压缩与归档 56

第1章 Linux 系统基础知识

1.1 Linux 系统

1.1.1 关于 Linux 内核的起源

1990 年 Linus Torvalds 开始接触 Unix 操作系统，1991 年 8 月 25 日 Linus Torvalds 在网络上发布了 Linux 内核原始代码。1994 年 3 月 14 日发布了 Linux 内核 1.0 版本。

图 1.1.1 Linus Toravalds 本人

2005 年，为了更好的管理 Linux 内核代码，Linus Torvalds 本人自己动手开发了大名鼎鼎的版本控制工具 git。

Linux 的命令问题

从技术层面讲，Linux 只是操作系统的内核名字，没别的含义。当然内核非常重要，因为有它操作系统才能运行起来，但它并不能构成一个完备的操作系统。

Richard Stallman 博士，自由软件运动创始人，自由软件基金会创办者，他创建了 GNU 工程，编写了第一版 GNU C 编译器（gcc），创立了 GNU 通用公共协议（the GPL）。

图 1.1.2 Richard Stallman 博士

他坚持把 Linux 称为“GNU/Linux”，为的是准确地反映 GNU 工程对 Linux 操作系统的贡献。

然而，尽管 GNU 项目早于 Linux 内核，项目的贡献应该得到极高的赞誉，但是把 GNU 用在 Linux 名字里，这对其他为 Linux 的发展做出重大贡献的程序员来说，就不公平了。

而且，我觉得要是叫也要叫“Linux/GNU”比较准确一些，因为内核会先启动，其他一切都运行在内核之上。

整个 Linux 生态系统，不只有 GNU 项目软件。在操作系统商界，好像喜欢使用单个词的名字，比如说 DOS，Windows，MacOS，所以大家习惯叫 Linux 而不是 GNU/Linux。

为什么使用命令行，你是否注意到，在电影中一个“超级黑客”坐在电脑前，从不摸一下鼠标，就能够在 30 秒内侵入到超安全的军用计算机中。这是因为电影制片人意识到，作为人类，本能地知道让计算机圆满完成工作的唯一途径，是用键盘来操纵计算机。

现在，大多数的计算机用户只是熟悉图形用户界面（GUI），并且产品供应商和此领域的学者会灌输给用户这样的思想，命令行界面（CLI）是过去使用的一种很恐怖的东西。这就很不幸，因为一个好的命令行界面，是用来和计算机进行交流沟通的非常有效的方式，正像人类社会使用文字互通信息一样。人们说，“图形用户界面让简单的任务更容易完成，而命令行界面使完成复杂的任务成为可能”，到现在这句话仍然很正确。

因为 Linux 是以 Unix 家族的操作系统为模型写成的，所以它继承了 Unix 丰富的命令行工具。事实上，很多人选择 Linux（而不是其他的系统，比如说 Windows NT）是因

为其可以使“完成复杂的任务成为可能”的强大的命令行界面。

对于 Linux 的启蒙教育，没有捷径可言。学习命令行充满挑战，而且很费气力。这并不是说 Linux 命令行很难学，而是它的知识量很大，不容易掌握。Linux 操作系统，差不多有数以千计的命令可供用户操作。由此可见，要给自己提个醒，命令行可不是轻轻松松就能学好的。

另一方面，学习 Linux 命令行会让你受益匪浅，给你极大的回报。如果你认为，现在你已经是高手了。别急，其实你还不知道什么才是真正的高手。不像其他一些计算机技能，一段时间之后可能就被淘汰了，命令行知识却不会落伍，你今天所学到的，在十年以后，都会有用处。命令行通过了时间的考验。

1.1.2 什么是 shell

一说到命令行，我们实际上指的是 shell。

shell 脚本解释程序，它接受从键盘输入的命令，然后把命令传递给操作系统去执行。几乎所有的 Linux 发行版都提供一个来自 GNU 项目名为 bash 的应用程序。“bash”是 “Bourne Again SHell”的首字母缩写。bash 是 Unix 上的 sh 的一个增强版。

当使用图形用户界面时，需要另一个和 shell 交互的叫做终端仿真器的程序。如果我们浏览一下桌面菜单，可能会找到一个。虽然在菜单里它可能都被简单地称为“terminal”，但是 KDE 用的是 konsole ，而 GNOME 则使用 gnome-terminal。还有其他一些终端仿真器可供 Linux 使用，但基本上它们都完成同样的事情，让我们能访问 shell。也许，你可能会因为附加的一系列花俏功能而喜欢上某个终端。启键盘按下 Ctrl+Alt+t 启动终端，一旦它运行起来，我们应该看到一行像这样的文字：

```
bkrc@bkrc:~ $
```

它通常包括你的用户名@主机名，紧接着当前工作目录和一个美元符号。如果提示符的最后一个字符是“#”，而不是“$”，那么这个终端会话就有超级用户权限。尝试在终端程序键入字符，在提示符下敲入一些乱七八糟的字符。

```
bkrc@bkrc:/$ bkrcabc
bash: bkrcabc: command not found
```

因为这个命令没有任何意义，所以 shell 会提示错误信息 command not found，并让我们再试一下。

如果按下上箭头按键，我们会看到刚才输入的命令“bkrcacb”重新出现在提示符

之后。这就叫做命令历史。许多 Linux 发行版默认保存最后输入的 500 个命令。按下下箭头按键，先前输入的命令就消失了。可借助上箭头按键，来获得上次输入的命令。现在试着使用左右箭头按键。看一下怎样把光标定位到命令行的任意位置？通过使用箭头按键，使编辑命令变得轻松些。

注意：不要在一个终端窗口里使用 Ctrl+c 和 Ctrl+v 快捷键来执行拷贝和粘贴操作。它们不起作用。对于 shell 来说，这两个控制代码有着不同的含义，它们在早于 Microsoft Windows（定义复制粘贴的含义）许多年之前就赋予了不同的意义。

运行一些简单的命令。

现在，我们学习了怎样输入命令，从一些简单的命令开始吧，第一个命令是 date。这个命令显示系统当前时间和日期。

```
bkrc@bkrc:~$ date
Tue 19 Nov 2019 03:29:50 AM CST
```

第二个命令是 cal，它默认显示当前月份的日历。

```
bkrc@bkrc:~$ cal
     November 2019
Su Mo Tu We Th Fr Sa
                    1  2
 3  4  5  6  7  8  9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30
```

第三个命令是 df，用于查看磁盘剩余空间的数量，加参数-h 以人性化显示，几乎大部分类 Unix 操作系统都支持这个命令。

```
bkrc@bkrc:~$ df -h
Filesystem          Size   Used Avail Use% Mounted on
/dev/root            29G   8.6G    20G   32% /
devtmpfs            1.8G      0   1.8G    0% /dev
tmpfs               1.9G      0   1.9G    0% /dev/shm
tmpfs               1.9G   8.5M   1.9G    1% /run
tmpfs               5.0M   4.0K   5.0M    1% /run/lock
tmpfs               1.9G      0   1.9G    0% /sys/fs/cgroup
/dev/mmcblk0p1     253M    52M   201M   21% /boot
tmpfs               386M      0   386M    0% /run/user/1000
```

第四个命令是 free，用于显示空现内存的大小

```
bkrc@bkrc:~$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	3.8Gi	159Mi	3.4Gi	10Mi	255Mi	3.5Gi
Swap:	99Mi	0B	99Mi			

可以看到系统总的物理内存总共 3.8GB，已使用 159M，剩余 3.4G 内存空间可用。此外还启用了交换分区 Swap，交换分区大小为 100M，已使用 0M，剩余 100M 可用。通常在物理内存使用紧张的情况下，才会发生内存交换。我们可以通过关闭终端仿真器窗口，或者是在 shell 提示符下输入 exit 命令来终止一个终端会话。

1.1.3 幕后控制台

即使终端仿真器没有运行，在后台仍然有几个终端会话运行着。它们叫做虚拟终端或者是虚拟控制台。在大多数 Linux 发行版中，这些终端会话都可以通过按 Ctrl+Alt+F1 到 Ctrl+Alt+F6 访问。当一个会话被访问的时候，它会显示登录提示框，我们需要输入用户名和密码。要从一个虚拟控制台转换到另一个，按下 Alt 和 F1-F6(中的一个)。返回图形桌面，按下 Alt-F7。

这在特殊情况下，将非常有用。如当错误的修改 Linux 的配置文件后导致系统无法启动桌面，则可以通过进入虚拟终端，将错误修复。

1.2 Linux 常用基础命令（一）

1.2.1 文件系统中跳转

pwd — 打印出当前工作目录名

ls — 列出目录内容

cd — 切换工作目录

类似于 Windows，一个“类 Unix”的操作系统，比如说 Linux，以分层目录结构来组织所有文件。

这就意味着所有文件组成了一棵树型目录

这个目录树可能包含文件和其它的目录。文件系统中的第一级目录称为根目录。

根目录包含文件和子目录，子目录包含更多的文件和子目录，由图形化文件管理器显示的文件系统树，如图 1.2.1 所示:

图 1.2.1 目录树

大多数人都可能熟悉如图 1.2.1 所示描述文件系统树的图形文件管理器。注意，通常这是一棵倒置的树，也就是说，树根在最上面，而各个枝干在下面展开。

把文件系统想象成一个迷宫形状，就像一棵倒立的大树，我们站在迷宫的中间位置。在任意时刻，我们处于一个目录里面，我们能看到这个目录包含的所有文件，以及通往上面目录（父目录）的路径，和下面的各个子目录。我们所在的目录则称为当前工作目录。我们使用 pwd（print working directory(的缩写)）命令，来显示当前工作目录。

```
bkrc@bkrc:~$ pwd
/home/bkrc
```

当我们首次登录系统（或者启动终端仿真器会话）后，当前工作目录是我们的家目录。每个用户都有他自己的家目录，当用户以普通用户的身份操控系统时，家目录是唯一允许用户写入文件的地方。

列出一个目录包含的文件及子目录，使用 ls 命令。ls 可能是用户最常使用的命令了，这自有它的道理。通过它，我们可以知道目录的内容，以及各种各样重要文件和目录的属性。正如我们已经见到的，只要简单地输入 ls 就能看到在当前目录下所大多数命令使用的选项，是由一个中划线加上一个字符组成，例如，“-l”，但是许多命令，包括来自于 GNU 项目的命令，也支持长选项，长选项由两个中划线加上一个字组成。当然，许多命令也允许把多个短选项串在一起使用。下面这个例子，ls 命令有两个选项，“l”选项产生长格式输出，“t”选项按文件修改时间的先后来排序。

```
bkrc@bkrc:/$ ls -lt /usr
total 92
drwxr-xr-x     2 root root 12288 Nov 18 10:59 sbin
drwxr-xr-x 233 root root 12288 Nov 18 10:59 share
drwxr-xr-x    84 root root    4096 Nov 18 10:59 lib
drwxr-xr-x     2 root root 36864 Nov 18 10:58 bin
drwxr-xr-x    48 root root 12288 Nov    2 12:20 include
drwxr-xr-x     3 root root    4096 Sep 26 08:16 src
drwxr-xr-x     3 root root    4096 Sep 26 08:15 libexec
drwxr-xr-x    10 root root    4096 Sep 26 08:05 local
drwxr-xr-x     2 root root    4096 Sep    9 21:42 games
```

加上长选项 “--reverse”，则结果会以相反的顺序输出：

```
bkrc@bkrc:/$ ls -lt --reverse /usr
total 92
drwxr-xr-x     2 root root    4096 Sep    9 21:42 games
drwxr-xr-x    10 root root    4096 Sep 26 08:05 local
drwxr-xr-x     3 root root    4096 Sep 26 08:15 libexec
drwxr-xr-x     3 root root    4096 Sep 26 08:16 src
drwxr-xr-x    48 root root 12288 Nov    2 12:20 include
drwxr-xr-x     2 root root 36864 Nov 18 10:58 bin
drwxr-xr-x    84 root root    4096 Nov 18 10:59 lib
drwxr-xr-x 233 root root 12288 Nov 18 10:59 share
drwxr-xr-x     2 root root 12288 Nov 18 10:59 sbin
```

ls 命令有大量的选项。图 2.2.2 列出了最常使用的选项。

选项	长选项	描述
-a	--all	列出所有文件，甚至包括文件名以圆点开头的默认会被隐藏的隐藏文件。
-d	--directory	通常，如果指定了目录名，ls 命令会列出这个目录中的内容，而不是目录本身。 把这个选项与 -l 选项结合使用，可以看到所指定目录的详细信息，而不是目录中的内容。
-F	--classify	这个选项会在每个所列出的名字后面加上一个指示符。例如，如果名字是 目录名，则会加上一个'/'字符。
-h	--human-readable	当以长格式列出时，以人们可读的格式，而不是以字节数来显示文件的大小。
-l		以长格式显示结果。
-r	--reverse	以相反的顺序来显示结果。通常，ls 命令的输出结果按照字母升序排列。
-S		命令输出结果按照文件大小来排序。
-t		按照修改时间来排序。

图 1.2.2 ls 命令选项

最后让我们来深入研究长格式输出“-l”选项导致 ls 的输出结果以长格式输出。这种格式包含大量的有用信息。

```
lrwxrwxrwx 1 root root        12 Feb 14   2019 initctl -> /run/initctl
drwxr-xr-x 4 root root       120 Nov 19 03:17 input
crw-r--r-- 1 root root       1,   11 Nov 19 03:17 kmsg
lrwxrwxrwx 1 root root        28 Feb 14   2019 log -> /run/systemd/journal/dev-log
brw-rw---- 1 root disk        7,    0 Nov 19 03:17 loop0
-rw-r--r-- 1 pi pi     82 Nov   2 13:23 tpu.sh
```

ls 长格式列表的字段含义如图 1.2.3 所示：

字段	含义
-rw-r--r--	对于文件的访问权限。第一个字符指明文件类型。在不同类型之间， 开头的“ - ”说明是一个普通文件，“d”表明是一个目录。其后三个字符是文件所有者的 访问权限，再其后的三个字符是文件所属组中成员的访问权限.
1	文件的硬链接数目
root	文件所有者的用户名。
root	文件所属用户组的名字。
32059	以字节数表示的文件大小。
2019-11-03 11:05	上次修改文件的时间和日期。
oo-cd-cover.odf	文件名。

图 1.2.3 ls 长格式字段含义

要更改工作目录（此刻，我们站在树形迷宫里面），使用 cd 命令。输入 cd,

然后输入你想要去的工作目录的路径名。路径名就是沿着目录树的分支

到达想要的目录期间所经过的路线。路径名可通过两种方式来指定，一种是绝对路径，另一种是相对路径。我们先来介绍绝对路径。绝对路径开始于根目录，紧跟着目录树的一个个分支，一直到达所期望的目录或文件。

例如，你的系统中有一个目录，大多数系统程序都安装在这个目录下。这个目录的路径名是/usr/bin。它意味着从根目录（用开头的“/”表示）开始，有一个叫"usr"的目录包含了目录"bin"。

```
bkrc@bkrc:/usr/bin$ cd
bkrc@bkrc:~$ cd /usr/bin/
bkrc@bkrc:/usr/bin$ pwd
/usr/bin
bkrc@bkrc:/usr/bin$ cd
bkrc@bkrc:~$
```

当把工作目录转到/usr/bin 目录下，可以注意到 shell 提示符发生了改变，通常终端提示符自动显示当前工作目录名。

绝对路径从根目录开始，直到它的目的地，而相对路径开始于工作目录。

为了做到这个（用相对路径表示），我们在文件系统树中用一对特殊符号来表示相对位置。

这对特殊符号是"."(点)和".."(点点)。符号"."指的是工作目录，".."指的是工作目录的父目录。下面的例子说明怎样使用它。在终端将工作目录切换到/usr/bin。比方说我们想更改工作目录到/usr/bin 的父目录/usr。可以通过两种方法来实现。可以使用以下绝对路径名。

```
bkrc@bkrc:~$ cd /
bkrc@bkrc:/$ pwd
/
bkrc@bkrc:/$ cd /usr
bkrc@bkrc:/usr$ pwd
/usr
bkrc@bkrc:/usr$ cd /home/pi
bkrc@bkrc:~$ pwd
/home/bkrc
```

或者，也可以使用相对路径

```
bkrc@bkrc:~/Desktop$ pwd
/home/bkrc/Desktop
bkrc@bkrc:~/Desktop$ ls ../
coral        Desktop      Downloads    Music        Pictures     Templates        tpu.sh
wc
create_ap    Documents    MagPi        openvino.sh  Public       TPU-MobilenetSSD
Videos    web_pi
bkrc@bkrc:~/Desktop$ cd ../Documents/
bkrc@bkrc:~/Documents$ pwd
/home/pi/Documents
```

两种不同的方法，一样的结果。我们应该选哪一个呢？选输入量最少的那个！

有一件很重要的事，必须指出来。在几乎所有的情况下，你可以省略"./"。它是隐含的输入。

图 1.2.4 展示了 cd 命令常用的快捷键

快捷键	运行结果
cd	更改工作目录到你的家目录。
cd -	更改工作目录到先前的工作目录。
cd ~user_name	更改工作目录到用户家目录。例如, cd ~bob 会更改工作目录到用户"bob"的家目录。

图 1.2.4 cd 快捷键

1.2.2 关于文件名的重要规则

以“.”字符开头的文件名是隐藏文件。这仅表示，ls 命令不能列出它们，用 ls -a 命令就可以了。当你创建用户后，几个配置用户的隐藏文件被放置在你的家目录下。稍后，我们会仔细研究一些隐藏文件，来定制你的系统环境。另外，一些应用程序也会把它们的配置文件以隐藏文件的形式放在你的家目录下面。

文件名和命令名是大小写敏感的。文件名“File1”和“file1”是指两个不同的文件名。

Linux 没有“文件扩展名”的概念，不像其它一些系统。可以用你喜欢的任何名字来给文件起名。文件内容或用途由其它方法来决定。虽然类 Unix 的操作系统，不用文件扩展名来决定文件的内容或用途，但是有些应用程序会。

虽然 Linux 支持长文件名，文件名可能包含空格，标点符号，但标点符号仅限使用

“.”，“—”，下划线。最重要的是，不要在文件名中使用空格。如果你想表示词与词间的空格，用下划线字符来代替。过些时候，你会感激自己这样做。

```
bkrc@bkrc:/tmp/test$ ls
'bkrc bj'      bkrc_bj
bkrc@bkrc:/tmp/test$ cd bkrc_bj/
bkrc@bkrc:/tmp/test/bkrc_bj$ cd -
/tmp/test
bkrc@bkrc:/tmp/test$ cd "bkrc bj"/
bkrc@bkrc:/tmp/test/bkrc bj$ pwd
/tmp/test/bkrc bj
```

如果需要使用带空格的文件名，请使用双引号将其包裹。

1.3 Linux 常用基础命令（二）

1.3.1 操作文件和目录

cp — 复制文件和目录

mv — 移动/重命名文件和目录

mkdir — 创建目录

rm — 删除文件和目录

ln — 创建硬链接和符号链接

这五个命令属于最常使用的 Linux 命令之列。它们用来操作文件和目录。

现在，坦诚地说，用图形文件管理器来完成一些由这些命令执行的任务会更容易些。使用文件管理器，我们可以把文件从一个目录拖放到另一个目录、剪贴和粘贴文件、删除文件等等。那么，为什么还使用早期的命令行程序呢？

答案是命令行程序，功能强大灵活。虽然图形文件管理器能轻松地实现简单的文件操作，但是对于复杂的文件操作任务，则使用命令行程序比较容易完成。例如，怎样拷贝一个目录下所有的 HTML 文件，并且这些文件只在目标目录不存在或者版本比目标目录的文件更新的时候才拷贝。要完成这个任务，使用文件管理器相当难，使用命令行相当容易：

```
bkrc@bkrc:/tmp/test$ cp -u *.html /destination
```

参数-u 表示只有在源文件比目标文件新才复制。

在开始使用命令之前，我们需要介绍一个使命令行如此强大的 shell 特性。因为 shell 频繁地使用文件名，shell 提供了特殊字符来帮助你快速指定一组文件名。这些特殊字符叫做通配符。

使用通配符（也以文件名代换著称）允许你依据字符的组合模式来选择文件名。图 1.3.1 列出这些通配符以及它们所选择的对象：

通配符	意义
*	匹配任意多个字符（包括零个或一个）
?	匹配任意一个字符（不包括零个）
[characters]	匹配任意一个属于字符集中的字符
[!characters]	匹配任意一个不是字符集中的字符
[[:class:]]	匹配任意一个属于指定字符类中的字符

图 1.3.1 通配符

接受文件名作为参数的任何命令，都可以使用通配符。借助通配符，可构建复杂的文件名匹配规则，图 1.3.2 是一些类型匹配的范例：

模式	匹配对象
*	所有文件
g*	文件名以“g”开头的文件
b*.txt	以"b"开头，中间有零个或任意多个字符，并以".txt"结尾的文件
Data???	以“Data”开头，其后紧接着3个字符的文件
[abc]*	文件名以"a","b",或"c"开头的文件
BACKUP.[0-9][0-9][0-9]	以"BACKUP."开头，并紧接着3个数字的文件
[[:upper:]]*	以大写字母开头的文件
[![:digit:]]*	不以数字开头的文件
*[[:lower:]123]	文件名以小写字母结尾，或以“1”，“2”，或“3”结尾的文件

图 1.3.2 通配符范例

mkdir 命令是用来创建目录的。它这样使用：

```
mkdir directory...
```

注意表示法:… 在描述一个命令时（如上所示），当有三个圆点跟在一个命令的参数后面，这意味着那个参数可以重复，就像这样：

```
mkdir dir1
```

会创建一个名为"dir1"的目录，而

```
mkdir dir1 dir2 dir3
```

会创建三个目录，名为 dir1, dir2, dir3。

```
bkrc@bkrc:/tmp/test$ mkdir dir
bkrc@bkrc:/tmp/test$ ls
dir
bkrc@bkrc:/tmp/test$ mkdir dir1 dir2 tmp
bkrc@bkrc:/tmp/test$ ls -l
total 16
drwxr-xr-x 2 bkrc bkrc 4096 Nov 19 04:23 dir
drwxr-xr-x 2 bkrc bkrc 4096 Nov 19 04:24 dir1
drwxr-xr-x 2 bkrc bkrc 4096 Nov 19 04:24 dir2
drwxr-xr-x 2 bkrc bkrc 4096 Nov 19 04:24 tmp
```

创建多级目录时，可使用参数 p，当多级目录的上级目录不存在时，会自动创建。

```
bkrc@bkrc:/tmp/test$ mkdir ch/bj/bkrc
mkdir: cannot create directory ‘ch/bj/bkrc’: No such file or directory
bkrc@bkrc:/tmp/test$ mkdir -p ch/bj/bkrc
bkrc@bkrc:/tmp/test$ ls -l
total 4
drwxr-xr-x 3 pi pi 4096 Nov 19 04:25 ch
bkrc@bkrc:/tmp/test$ ls -l ch/bj/
total 4
drwxr-xr-x 2 pi pi 4096 Nov 19 04:25 bkrc
```

cp 命令，复制文件或者目录。它有两种使用方法：

```
cp item1 item2
```

复制单个文件或目录"item1"到文件或目录"item2"，和：

```
cp item... directory
```

复制多个项目（文件或目录）到一个目录下。

这里列举了 cp 命令一些有用的选项（短选项和等效的长选项）：

选项	意义
-a, --archive	复制文件和目录，以及它们的属性，包括所有权和权限。 通常，副本具有用户所操作文件的默认属性。
-i, --interactive	在重写已存在文件之前，提示用户确认。如果这个选项不指定， cp 命令会默认重写文件。
-r, --recursive	递归地复制目录及目录中的内容。当复制目录时， 需要这个选项（或者-a 选项）。
-u, --update	当把文件从一个目录复制到另一个目录时，仅复制 目标目录中不存在的文件，或者是文件内容新于目标目录中已经存在的文件。
-v, --verbose	显示翔实的命令操作信息

图 1.3.2 cp 命令参数

命令	运行结果
cp file1 file2	复制文件 file1 内容到文件 file2。如果 file2 已经存在， file2 的内容会被 file1 的内容重写。如果 file2 不存在，则会创建 file2。
cp -i file1 file2	这条命令和上面的命令一样，除了如果文件 file2 存在的话，在文件 file2 被重写之前， 会提示用户确认信息。
cp file1 file2 dir1	复制文件 file1 和文件 file2 到目录 dir1。目录 dir1 必须存在。
cp dir1/* dir2	使用一个通配符，在目录 dir1 中的所有文件都被复制到目录 dir2 中。 dir2 必须已经存在。
cp -r dir1 dir2	复制目录 dir1 中的内容到目录 dir2。如果目录 dir2 不存在， 创建目录 dir2，操作完成后，目录 dir2 中的内容和 dir1 中的一样。 如果目录 dir2 存在，则目录 dir1 (和目录中的内容)将会被复制到 dir2 中。

图 1.3.3 cp 命令示例

mv 命令可以执行文件移动和文件命名任务，这依赖于你怎样使用它。任何一种情况下，完成操作之后，原来的文件名不再存在。mv 使用方法与 cp 很相像：

```
mv item1 item2
```

把文件或目录“item1”移动或重命名为“item2”，或者：

```
mv item... directory
```

把一个或多个条目从一个目录移动到另一个目录中。

如何区分 mv 命令是改名还是剪切，可以根据参数路径来区分，如果参数路径都是在同一路径下则是改名，如果不是在同一路径下就是剪切加改名。

```
bkrc@bkrc:/tmp/test$ ls
```

```
bkrc.txt    ch
bkrc@bkrc:/tmp/test$ mv bkrc.txt bkrc.md        mv 在这里的作用是改名
bkrc@bkrc:/tmp/test$ ls
bkrc.md    ch
bkrc@bkrc:/tmp/test$ mv bkrc.md ch/bj/bkrc/1.txt      mv 在这里的作用为剪切加改名
bkrc@bkrc:/tmp/test$ ls
ch
bkrc@bkrc:/tmp/test$ ls -l ch/bj/bkrc/1.txt
-rw-r--r-- 1 pi pi 0 Nov 19 04:28 ch/bj/bkrc/1.txt
```

mv 和 cp 命令很多参数都是一样的

选项	意义
-i --interactive	在重写一个已经存在的文件之前，提示用户确认信息。**如果不指定这个选项，mv 命令会默认重写文件内容。**
-u --update	当把文件从一个目录移动另一个目录时，只是移动不存在的文件，或者文件内容新于目标目录相对应文件的内容。
-v --verbose	当操作 mv 命令时，显示翔实的操作信息。

图 1.3.4 mv 参数

rm 命令用来移除（删除）文件和目录：

```
rm item...
```

"item"代表一个或多个文件或目录。

rm 常用参数

选项	意义
-i, --interactive	在删除已存在的文件前，提示用户确认信息。**如果不指定这个选项，rm 会默默地删除文件**
-r, --recursive	递归地删除文件，这意味着，如果要删除一个目录，而此目录又包含子目录，那么子目录也会被删除。要删除一个目录，必须指定这个选项。
-f, --force	忽视不存在的文件，不显示提示信息。这选项覆盖了"--interactive"选项。
-v, --verbose	在执行 rm 命令时，显示翔实的操作信息。

图 1.3.5 rm 参数

rm 命令实例

命令	运行结果
rm file1	默默地删除文件
rm -i file1	除了在删除文件之前，提示用户确认信息之外，和上面的命令作用一样。
rm -r file1 dir1	删除文件 file1, 目录 dir1，及 dir1 中的内容。
rm -rf file1 dir1	同上，除了如果文件 file1，或目录 dir1 不存在的话，rm 仍会继续执行。

图 1.3.6 rm 实例

在使用 rm 命令时一定要小心，在 Linux 系统中，没有复原命令。一旦你使用 rm 删除一些东西，它就消失了。尤其要小心通配符，思考一下这个经典的例子。假如说，你只想删除一个目录中的所有的 HTML 文件

```
rm *.html
```

这是正确的，如果你不小心在“*”和“.html”之间多输入了一个空格，就像这样：

```
rm * .html
```

这个 rm 命令会删除目录中的所有文件，还会抱怨没有文件叫做“.html”。

```
bkrc@bkrc:/tmp/test$ ls
bkrc.html    index.html    video.html
bkrc@bkrc:/tmp/test$ rm * .html
rm: cannot remove '.html': No such file or directory
bkrc@bkrc:/tmp/test$ ls
bkrc@bkrc:/tmp/test$
```

当你使用带有通配符的 rm 命令时（除了仔细检查输入的内容外），先用 ls 命令来测试通配符。这会让你看到将要被删除的文件是什么。然后按下上箭头按键，重新调用。

ln 命令既可创建硬链接，也可以创建符号链接。可以用其中一种方法来使用它：

```
ln file link
```

创建硬链接：

```
ln -s item link
```

创建符号链接，"item" 可以是一个文件或是一个目录。

与更加现代的符号链接相比，硬链接是最初 Unix 创建链接的方式。每个文件默认会有一个硬链接，这个硬链接给予文件名字。我们每创建一个硬链接，就为一个文件创

建了一个额外的目录项。

硬链接有两个重要局限性：

1. 一个硬链接不能关联它所在文件系统之外的文件。这是说一个链接不能关联与链接本身不在同一个磁盘分区上的文件。

2. 一个硬链接不能关联一个目录。

一个硬链接和文件本身没有什么区别。不像符号链接，当你列出一个包含硬链接的目录内容时，你会看到没有特殊的链接指示说明。当一个硬链接被删除时，这个链接被删除，但是文件本身的内容仍然存在（这是说，它所占的磁盘空间不会被重新分配），直到所有关联这个文件的链接都删除掉。

创建符号链接是为了克服硬链接的局限性。符号链接生效，是通过创建一个特殊类型的文件，这个文件包含一个关联文件或目录的文本指针。在这一方面，它们和 Windows 的快捷方式差不多，当然符号链接早于 Windows 的快捷方式很多年。

一个符号链接指向一个文件，而且这个符号链接本身与其它的符号链接几乎没有区别。例如，如果你往一个符号链接里面写入东西，那么相关联的文件也被写入。然而，当你删除一个符号链接时，只有这个链接被删除，而不是文件自身。如果先于符号链接删除文件，这个链接仍然存在，但是不指向任何东西。在这种情况下，这个链接被称为坏链接。ls 命令会以不同的颜色展示坏链接，比如说红色，来显示它们的存在。

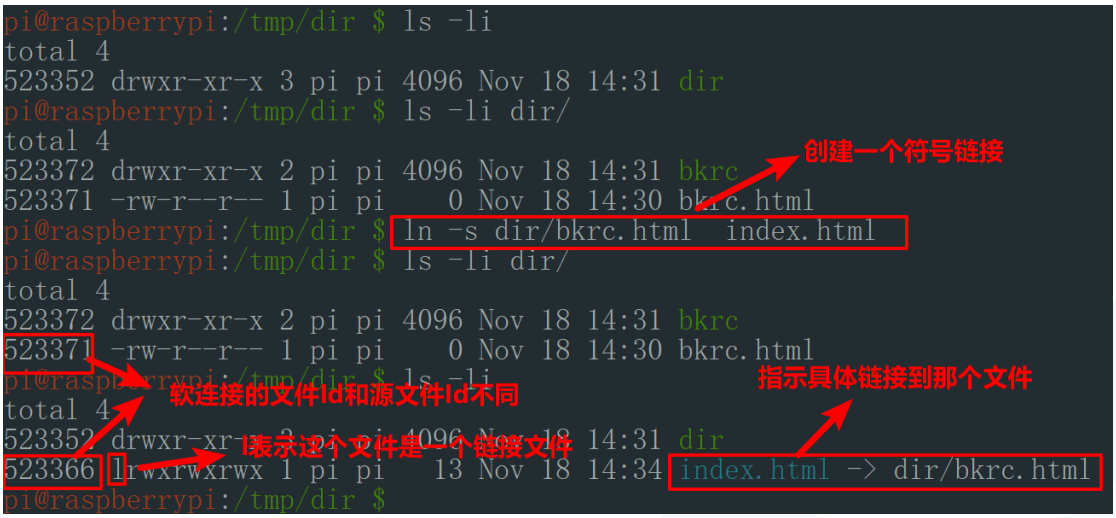

图 1.3.7 符号链接

删除链接文件所指的源文件，链接文件依旧存在，只不过该链接文件是坏链接。

```
pi@raspberrypi:/tmp/dir $ ls -li
total 4
523352 drwxr-xr-x 3 pi pi 4096 Nov 18 14:31 dir
523366 lrwxrwxrwx 1 pi pi   13 Nov 18 14:34 index.html -> dir/bkrc.html
pi@raspberrypi:/tmp/dir $ rm dir/bkrc.html
pi@raspberrypi:/tmp/dir $ ls -li
total 4
523352 drwxr-xr-x 3 pi pi 4096 Nov 18 14:49 dir
523366 lrwxrwxrwx 1 pi pi   13 Nov 18 14:34 index.html -> dir/bkrc.html
pi@raspberrypi:/tmp/dir $
```

链接文件源文件存在时

链接文件源文件不存在时
以红色显示，代表是坏链接

图 1.3.8 坏符号链接

1.3.2 浏览文件内容

cat -- 查看文件内容

head -- 查看文件的开头几行

tail -- 查看文件的结尾几行

more -- 浏览文本内容

less -- 浏览文本内容

cat 命令可用来查看纯文本文件中的内容，cat 命令会将文本文件中内容全部显示在屏幕上。

```
bkrc@bkrc:/tmp/test$ cat hello.py
#!/usr/bin/env python3
import numpy as np
zers = np.zeros((3,3), np.uint8)
print(zers)
```

对于小的文本文件可以使用 cat 命令查看，但 cat 命令不适合浏览大量的文本文件。

有时候，我们只需要查看文件开头几行或者文件末尾几行，使用 head 命令和 tail 命令就非常方便。

```
bkrc@bkrc:/tmp/test$ head -5 ~/.bashrc
# ~/.bashrc: executed by bash(1) for non-login shells.
# see /usr/share/doc/bash/examples/startup-files (in the package bash-doc)
# for examples

# If not running interactively, don't do anything
bkrc@bkrc:/tmp/test$ tail -5 ~/.bashrc
#cd /home/pi/Desktop/servo_face/ && python3 servo_face.py
```

```
PYTHONPATH=/home/pi/web_pi:/opt/intel/openvino/python/python3.7:/opt/intel/openvino
/python/python3:/opt/intel/openvino/deployment_tools/model_optimizer:
```

more 和 less 命令都可以用来浏览文本内容，纵观 Linux 系统，有许多人类可读的文本文件。more 和 less 命令为我们检查文本文件提供了方便。

more 命令的用法和 cat 非常相似，cat 命令读取整个文件内容然后全部显示刷到屏幕上。非常的不利于阅读，more 命令则是将文件内容划分为页，使用空格键下翻页,使用回车键下翻行。但 more 命令不支持回滚。而 less 命令则弥补了 more 命令不能回滚的遗憾。

less 命令是一个非常好用的命令，less 命令允许你前后滚动文件，例如，要查看当前系统中全部用户。

```
less /ect/passwd
```

如果文件内容一页显示不完，那么可以通过键盘命令来滚动文件，退出按“q”键。

图 1.3.9 列出了 less 命令最常用的键盘命令。

命令	行为
Page UP or b	向上翻滚一页
Page Down or space	向下翻滚一页
UP Arrow	向上翻滚一行
Down Arrow	向下翻滚一行
G	移动到最后一行
1G or g	移动到开头一行
/charaters	向前查找指定的字符串
n	向前查找下一个出现的字符串，这个字符串是之前所指定查找的
h	显示帮助屏幕
q	退出 less 程序

图 1.3.9 less 键盘命令

日常推荐使用 less，因为 less 和 more 都能以逐页的方式轻松浏览长文本文档，但是 more 只能前向翻页，而 less 允许后翻页。此外 less 还有很多其他特性。

1.3.3 流重定向

在这一小节中，我们来介绍可能是命令行中最酷的特性。它叫做 I/O 重定向。"I/O"代表输入/输出，通过这个工具，你可以重定向命令的输入输出，命令的输入来自文件，而输出也存到文件。也可以把多个命令连接起来组成一个强大的命令管道。为了展示这个功能，将介绍以下命令：

cat -- 连接文件

sort -- 排序文本行

grep -- 打印匹配行

wc -- 打印文件中换行符，字，和字节个数

tee -- 从标准输入读取数据，并同时写到标准输出和文件

到目前为止，我们用到的许多程序都会产生某种输出。这种输出，经常由两种类型组成。第一，程序运行结果；这是说，程序要完成的功能。第二，我们得到状态和错误信息，这些告诉我们程序进展。如果我们观察一个命令，像 ls，会看到它的运行结果和错误信息显示在屏幕上。

在 Linux 系统中比如 ls,它们的运行结果会输送到一个叫做标准输出的特殊文件（一般情况下用 stdout 表示），而它们的状态信息则输送到另一个叫做标准错误的文件（stderr）。默认情况下，标准输出和标准错误都连接到屏幕，而不是保存到磁盘文件。除此之外，许多程序从一个叫做标准输入（stdin）的设备得到输入，默认情况下，标准输入连接到键盘。

I/O 重定向允许我们来重定义标准输出的地点。我们使用">"重定向符后接文件名将标准输出重定向到除屏幕以外的另一个文件。为什么我们要这样做呢？因为有时候把一个命令的运行结果存储到一个文件很有用处。例如，我们可以告诉 shell 把 ls 命令的运行结果输送到文件 ls_output.txt 中去，由文件代替屏幕。

```
ls -l > /tmp/ls_output.txt
```

使用 cat 命令阅读这个文本文件，我们会看到 ls_output.txt 的确包含 ls 命令的执行结果。

```
bkrc@bkrc:/tmp/test$ ls / > ls_output.txt
bkrc@bkrc:/tmp/test$ cat ls_output.txt
bin
boot
```

```
dev
etc
home
lib
```

现在，重复我们的重定向测试，但这次有改动。我们把目录换成一个不存在的目录。

```
bkrc@bkrc:/tmp$ ls -l /bin/usr > /tmp/ls_output.txt
ls: cannot access '/bin/usr': No such file or directory
```

收到一个错误信息。这讲得通，因为指定了一个不存在的目录/bin/usr,但是为什么这条错误信息显示在屏幕上而不是被重定向到文件 ls-output.txt？答案是，ls 程序不把它的错误信息输送到标准输出。反而，像许多写得不错的 Linux 程序，ls 把错误信息送到标准错误。因为我们只是重定向了标准输出，而没有重定向标准错误，所以错误信息被送到屏幕。马上，我们将知道怎样重定向标准错误，但是首先看一下我们的输出文件发生了什么事情。

```
bkrc@bkrc:/tmp$ ls -l ls_output.txt
-rw-r--r-- 1 pi pi 0 Nov 19 05:04 ls_output.txt
```

文件长度为零！这是因为，当我们使用“>”重定向符来重定向输出结果时，目标文件总是从开头被重写。因为我们 ls 命令没有产生运行结果，只有错误信息，重定向操作开始重写文件，然后由于错误而停止，导致文件内容清空。简单地使用重定向符，没有命令在它之前，这会清空一个已存在文件的内容或是创建一个新的空文件。

```
bkrc@bkrc:/tmp/test$ ls
bkrc@bkrc:/tmp/test$ > output.txt
bkrc@bkrc:/tmp/test$ ls -l
total 0
-rw-r--r-- 1 pi pi 0 Nov 19 05:07 output.txt
```

所以，怎样才能把重定向结果追加到文件内容后面，而不是从开头重写文件？为实现这个目的，我们使用"><"重定向符，像这样：

```
bkrc@bkrc:/tmp/test$ ls -l / >> /tmp/ls_output.txt
bkrc@bkrc:/tmp/test$ ls -l /usr >> /tmp/ls_output.txt
bkrc@bkrc:/tmp/test$ ls -l /tmp/ls_output.txt
-rw-r--r-- 1 pi pi 1391 Nov 19 05:09 /tmp/ls_output.txt
```

使用"><"操作符，将导致输出结果添加到文件内容之后。如果文件不存在，文件会被创建。我们重复执行命令三次，导致输出文件大小是原来的三倍。

```
bkrc@bkrc:/tmp/test$ ls -l /usr >> ls_output.txt
```

```
bkrc@bkrc:/tmp/test$ ls -l ls_output.txt
-rw-r--r-- 1 pi pi 456 Nov 19 05:11 ls_output.txt
bkrc@bkrc:/tmp/test$ ls -l /usr >> ls_output.txt
bkrc@bkrc:/tmp/test$ ls -l /usr >> ls_output.txt
bkrc@bkrc:/tmp/test$ ls -l ls_output.txt
```

标准错误重定向没有专用的重定向操作符。为了重定向标准错误，需要添加文件描述符，请记住在 Linux 中前三个文件标准流分别是标准输入、输出和错误。对应 shell 内部的文件描述符 0（标准输入）、1（标准输出）和 2（标准错误）。

因为标准错误和文件描述符 2 一样，因此可以使用下面表示方法来重定向标准错误。

文件描述符"2"，紧挨着放在重定向操作符之前，来执行重定向标准错误到文件 ls_error.txt 任务。

```
bkrc@bkrc:/tmp/test$ ls -l /bin/usr > error_output.txt
ls: cannot access '/bin/usr': No such file or directory
bkrc@bkrc:/tmp/test$ ls -l /bin/usr 2> error_output.txt
bkrc@bkrc:/tmp/test$ cat error_output.txt
ls: cannot access '/bin/usr': No such file or directory
```

可能有这种情况，我们希望捕捉一个命令的所有输出到一个文件。为了完成这个，我们必须同时重定向标准输出和标准错误。有两种方法来完成任务。

第一种，首先重定向标准输出到文件 ls_output.txt，然后重定向文件描述符 2（标准错误）到文件描述符 1（标准输出）使用表示法 2>&1。

```
ls -l /bin/usr > ls-output.txt 2>&1
```

注意，第一种方法对重定向流顺序的安排必须是标准错误流重定向到标准输出，不然上述的例子将无效。

第二种方法，在现代版的 bash 程序中，使用一种更为简洁的方法来执行这种联合重定向。

```
bkrc@bkrc:/tmp/test$ ls /usr /bin/usr &> output.txt
bkrc@bkrc:/tmp/test$ cat output.txt
ls: cannot access '/bin/usr': No such file or directory
/usr:
bin
games
include
lib
libexec
```

```
local
sbin
share
src
```

在上面这个例子中，仅使用单单一个表示法&>来重定向标准输出和错误到文件 ls-output.txt。

在 Linux 系统中一直秉承着一句这样的话，“没有消息就是好消息”，有些时候“沉默是金”，我们不想要一个命令的输出结果，只想把它们扔掉。这种情况尤其适用于错误和状态信息。系统通过重定向输出结果到一个叫做"/dev/null"的特殊文件，为我们提供了解决问题的方法。这个文件是系统设备，叫做位存储桶，它可以接受输入，并且对输入不做任何处理。为了隐瞒命令错误信息，我们可以这么做。

```
ls -l    /bin/usr 2> /dev/null
```

在上一小节中，我们大致的介绍 cat 命令的使用，现在我将结合标准输入输出流来更详细的将探讨 cat 命令。

cat 命令读取一个或多个文件，然后复制它们到标准输出，就像这样：

```
cat file ...
```

实际上 cat 最常用于文件连接，比如我们下载了一个大型文件，这个文件被分离成很多小部分。最后我们想把这些小文件连接起来。

```
cat video.mpeg.00 video.mpeg.01 video.mpeg.02 > video.mpeg
```

很好的一个命令，但这和标准输入输出流有什么关系呢？没有任何关系。让我们尝试输入不带参数的 cat 命令。

```
cat
```

没有发生任何事情，程序好像挂掉了一样，但是它正在等待我们输入数据，试试输入一些数据，然后输入 Ctrl+ D（按住 Ctrl 键同时按下“d”），来告诉 cat，在标准输入中已经达到文件末尾（EOF）。

由于没有文件名参数，cat 复制标准输入到标准输出，所以我们看到文本行重复出现。我们可以使用这种行为来创建简短的文本文件。比方说，我们想创建一个叫做“bkrc.txt”的文件，可以这么做：

输入命令，其后输入要放入文件中的文本。记住，最后输入 Ctrl-d。通过使用这个命令，我们实现了世界上最低能的文字处理器！

```
bkrc@bkrc:/tmp/test$ cat >    bkrc.txt
```

```
bkrc
bkrc
Say Hello
```

看一下运行结果，我们使用 cat 来复制文件内容到标准输出。

```
bkrc@bkrc:/tmp/test$ cat bkrc.txt
bkrc
bkrc
Say Hello
```

现在我们知道 cat 怎样接受标准输入，除了文件名参数，让我们试着重定向标准输入：

```
cat < /tmp/cat_output.txt
```

使用“<”重定向操作符，我们把标准输入源从键盘改到文件 cat_output.txt。我们看到结果和传递单个文件名作为参数的执行结果一样。把这和传递一个文件名参数作比较，不是特别有意义，但它是用来说明把一个文件作为标准输入源。

使用管道操作符"|"（竖杠），一个命令的标准输出可以通过管道送至另一个命令的标准输入：

```
command1 | command2
```

为了说明管道符，我们需要一些命令。是否记得前面我们讲到过的 less 命令，现在我们用 less 来一页一页地显示任何命令的输出，命令把它的运行结果输送到标准输出：

```
ls /urs/bin | less
```

在实际应用中，会使用管道符将几个命令放在一起组成一个管道线，来完成对数据的复杂操作。以这种方式使用命令被称为过滤器。过滤器接受输入，以某种方式改变它，然后输出它。

sort 命令从标准输入接收数据，然后将排序结果输送到标准输出。我们想把目录/bin 和/usr/bin 中的可执行程序都联合在一起，再把它们排序，然后浏览执行结果：

```
ls /bin /usr/bin | sort | less
```

管道符会把上一条命令的执行结果输送到管道符后一个命令的输入中，即管道符会把一个命令的输出结果送到另一个命令的接收输入中。

wc（字计数）命令是用来显示文件所包含的行数、字数和字节数。例如：

```
ls | wc
# 输出 行数、字数、字节数
```

下面这个例子可以用来统计根目录下文件数量（不含子目录）

```
cd /
ls |wc -l
# 输出 20
```

grep 是个很强大的程序，用来找到文件中的匹配文本。这样使用 grep 命令：

```
grep pattern [file...]
```

举个例子，比如说我们想在程序列表中，找到文件名中包含单词“zip”的所有文件。可以这样写：

```
ls /bin /usr/bin | grep zip
```

有一些方便的选项："-i"使得 grep 在执行搜索时忽略大小写（通常，搜索是大小写敏感的），"-v"选项会告诉 grep 只打印不匹配的行。

在本小节的最后，我们来学习 tee 命令，tee 命令从 Stdin 读取数据，并同时输出到 Stdout 和文件。我们重复执行原来的例子，这次包含 tee 命令。在 grep 过滤管道线的内容之前，来捕捉整个目录列表到文件 ls.txt：

```
bkrc@bkrc:/tmp/test$ ls /bin /usr/bin/ | tee /tmp/ls_.txt |grep zip
bunzip2
bzip2
bzip2recover
gunzip
bkrc@bkrc:/tmp/test$ cat    /tmp/ls_.txt    |grep zip
bunzip2
bzip2
bzip2recover
gunzip
```

1.4 Linux 权限

1.4.1 用户和权限

在类 Unix 系统中，默认情况下，超级管理员为 root 用户，这一点和 Windows 下的管理员类似。具体的讲用户 UID 号为 0 的都是超级管理员，所以 root 这个用户的 ID 号为 0，关于这一点可以查看文件/etc/passwd 来佐证。

```
bkrc@bkrc:/tmp/test$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
```

Linux 是一个多用户，多任务的操作系统，在/etc/passwd 文件中，每一行表示一个 Linux 用户，每行用冒号分割，包含如下字段：

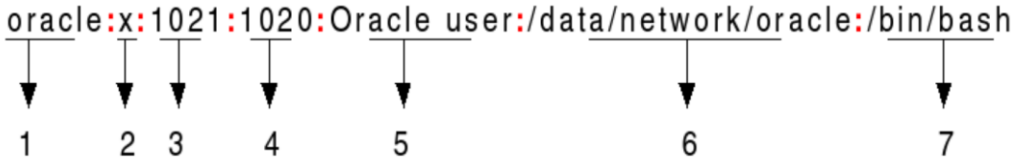

图 1.4.1 /etc/passwd 中的字段

1. 用户名：用户登录时使用。长度在 1 到 32 个字符之间。
2. 密码：x 字符表示加密的密码存储在/ etc / shadow 文件中。
3. 用户 ID（UID）：必须为每个用户分配一个用户 ID（UID）。UID 0（零）保留给根用户，UID 1-99 保留给其他预定义帐户。系统还为管理帐户和系统帐户/组保留了其他 UID 100-999。
4. 组 ID（GID）：主要组 ID（存储在/ etc / group 文件中）
5. 用户 ID 信息:注释字段。它允许您添加有关用户的其他信息,例如用户的全名，电话号码等。该字段由 finger 命令使用。
6. 主目录：用户登录时将位于的目录的绝对路径。如果此目录不存在，则用户目录将变为/
7. Command / shell：命令或外壳程序的绝对路径（/ bin / bash）。通常，这是一个外壳。请注意，它不必是 shell。

/etc/passwd 所有用户都可读，但只有 UID 为 0 的用户才具有写权限。

```
bkrc@bkrc:~$ ls -l /etc/passwd
-rw-r--r-- 1 root root 1691 Nov 18 10:59 /etc/passwd
```

另外普通用户的 UID 是从 1000 开始的，在开发平台中有一个普通用户 bkrc，这是系统

中的第一个普通用户，所以第 UID 为 1000。

```
bkrc@bkrc:~$ cat /etc/passwd |grep bkrc
bkrc:x:1000:1000:,,,:/home/bkrc:/bin/bash
```

当在不指定 UID 创建第二个普通用户时，那么创建的这名用户的 UID 为 1001,以此类推。

当用户创建帐户之后，系统会给用户分配一串数字，叫做用户 UID，然后，为了符合人类的习惯，这个 ID 映射到一个用户名。系统又会给这个用户分配一个原始的 GID（组 ID），这个 GID 可能属于另外的组。一般可以使用 id 命令来查询 UID 和 GID。

```
bkrc@bkrc:~$ id
uid=1000(pi) gid=1000(pi) groups=1000(pi),4(adm),20(dialout)…
```

可以看到 Linux 下每个用户必须属于一个原始组，同时用户又可以附加到多个其他组。就像在一家公司，一个员工入职时被分派到技术服务组，后来由于公司需要被加入到软件开发组，之后又陆续加入软件测试组，产品设计组。这名员工同时在多个小组里，应此比较大的权限和更重的责任。

1.4.2 读取，写入和执行权限

当用户创建了一个文件后，该文件所在的组就是该用户所在的组。让我们看看用户默认创建的文件具有怎样的权限。

```
bkrc@bkrc:/tmp/test$ cat > output.txt
Say Hello
bkrc@bkrc:/tmp/test$ ls -l output.txt
-rw-r--r-- 1 pi pi 10 Nov 19 06:46 output.txt
```

这里重点关注列表的前十个字符，它们描述了文件的属性，这十个字符的第一个字符表明文件类型。表 1.4.1 说明了一些常见的文件类型。

表 1.4.1 文件类型

属性	文件类型
-	一个普通文件
d	一个目录
l	一个符号链接。注意对于符号链接文件，剩余的文件属性总是"rwxrwxrwx"，而且都是 虚拟值。真正的文件属性是指符号链接所指向的文件的属性。
c	一个字符设备文件。这种文件类型是指按照字节流来处理数据的设备。 比如说终端机或者调制解调器

b	一个块设备文件。这种文件类型是指按照数据块来处理数据的设备，例如一个硬盘或者 CD-ROM 盘。

剩下的九个字符叫做文件模式，代表着文件所有者、文件组所有者和其他人的读、写和执行权限。如图 1.4.2 所示

图 1.4.2 权限属性

r、w 和 x 模式属性对文件和目录会产生以下影响：如表 1.4.2 权限属性所示

表 1.4.2 权限属性

属性	文件	目录
r	允许打开并读取文件内容。	允许列出目录中的内容，前提是目录必须设置了可执行属性（x）。
w	允许写入文件内容或截断文件。但是不允许对文件进行重命名或删除，重命名或删除是由目录的属性决定的。	允许在目录下新建、删除或重命名文件，前提是目录必须设置了可执行属性(x)。
x	允许将文件作为程序来执行，使用脚本语言编写的程序必须设置为可读才能被执行。	允许进入目录，例如：cd directory 。

权限属性示例，如表 1.4.3 所示：

表 1.4.3 权限属性示例

文件属性	含义
-rwx------	一个普通文件，对文件所有者来说可读、可写、可执行。其他人无法访问。
-rw-------	一个普通文件，对文件所有者来说可读可写。其他人无法访问。
-rw-r--r--	一个普通文件，对文件所有者来说可读可写，文件所有者的组成员可以读该文件，其他所有人都可以读该文件。
-rwxr-xr-x	一个普通文件，对文件所有者来说可读、可写、可执行。也可以被其他的所有人读取和执行。

-rw-rw----	一个普通文件，对文件所有者以及文件所有者的组成员来说可读可写。
lrwxrwxrwx	一个符号链接，符号链接的权限都是虚拟的，真实的权限应该以符号链接指向的文件为准。
drwxrwx---	一个目录，文件所有者以及文件所有者的组成员可以访问该目录，并且可以在该目录下新建、重命名、删除文件。
drwxr-x---	一个目录，文件所有者可以访问该目录，并且可以在该目录下新建、重命名、删除文件，文件所有者的组成员可以访问该目录，但是不能新建、重命名、删除文件。

更改文件或目录的模式（权限），可以利用 chmod 命令。注意只有文件的所有者或者超级用户才能更改文件或目录的模式。chmod 命令支持两种不同的方法来改变文件模式：八进制数字表示法或符号表示法。首先介绍八进制数字表示法。

八进制（以 8 为基数），类似的计数法还有二进制（以 2 为基数）、十六进制（以 16 为基数）。在二进制中计数看起来像这样：

```
0, 1, 10, 11, 100, 101, 110, 111, 1000, 1001, 1010, 1011...
```

十六进制中，使用数字 0 到 9，加上大写字母"A"到"F"来计数，逢 16 进一：

```
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, 10, 11, 12, 13...
```

我们可能比较熟悉二进制，和十六进制带啦的好处，但是八进制和十六进制相比有什么优势呢？ 答案是为了计数便利。

许多时候，在计算机中，一小部分数据以二进制的形式表示。以 RGB 颜色为例。大多数的计算机显示器，每个像素由三种颜色组成：8 位红色，8 位绿色，8 位蓝色。这样，下面 24 位数字表示蓝色：

```
010000110110111111001101
```

每个十六进制数字代表四个二进制。在八进制中，每个数字代表三个二进制数字。那么代表中蓝色的 24 位二进制能够压缩成 6 位十六进制数：

```
436FCD
```

因为十六进制中的两个数字对应二进制的 8 位数字，我们可以看到“43”代表红色通道值，“6F”代表绿色通道值，“CD”代表蓝色通道。通过八进制表示法，我们使用八进制数字来设置所期望的权限模式。因为每个八进制数字代表了 3 个二进制数字，这种对应关系，正好映射到用来存储文件模式所使用的方案上。下面展示所要表达的这种映射关系。

|0 | 000 | ---|

|1 | 001 | --x|

|2 | 010 | -w-|

|3 | 011 | -wx|

|4 | 100 | r--|

|5 | 101 | r-x|

|6 | 110 | rw-|

|7 | 111 | rwx|

通过使用 3 个八进制数字，我们能够设置文件所有者、用户组和其他人的权限：

```
bkrc@bkrc:/tmp/test$ > bkrc.txt
bkrc@bkrc:/tmp/test$ ls -l bkrc.txt
-rw-r--r-- 1 pi pi 0 Nov 19 17:19 bkrc.txt
bkrc@bkrc:/tmp/test$ chmod 600 bkrc.txt
bkrc@bkrc:/tmp/test$ ls -l bkrc.txt
-rw------- 1 pi pi 0 Nov 19 17:19 bkrc.txt
```

通过传递参数“600”，我们能够设置文件所有者的权限为读写权限，而删除用户组和其他人的所有权限。虽然八进制到二进制的映射看起来不方便，但通常只会用到一些常见的映射关系：7(rwx)，6(rw-)，5(r-x)，4(r--)，和 0(---)。

此外，chmod 命令支持一种符号表示法，来指定文件模式。符号表示法分为三部分：更改会影响谁，要执行哪个操作，要设置哪种权限。通过字符 “u”、“g”、“o”和“a”的组合来指定要影响的对象，如表 2.4.4 所示：

表 2.4.4 chmod 命令符号表示法

u	"user"的简写，意思是文件或目录的所有者。
g	用户组。
o	"others"的简写，意思是其他所有的人。
a	"all"的简写，是"u"，"g"和“o”三者的联合。

如果没有指定字符，则假定使用"all"。执行的操作可能是“＋”字符，表示加上一个权限，“－”，表示删掉一个权限，或者是“＝”，表示只有指定的权限可用，其它所有的权限被删除。权限由 “r”、“w”和 “x” 来指定。表 2.4.5 是一些符号表示法的实例：

表 2.4.5 符号表示法实例

u+x	为文件所有者添加可执行权限。
u-x	删除文件所有者的可执行权限。
+x	为文件所有者，用户组，和其他所有人添加可执行权限。等价于 a+x。
o-rw	除了文件所有者和用户组，删除其他人的读权限和写权限。
go=rw	给文件所属的组和文件所属者/组以外的人读写权限。如果文件所属组或其他人已经拥有执行的权限，执行权限将被移除。
u+x,go=rw	给文件拥有者执行权限并给组和其他人读和执行的权限。多种设定可以用逗号分开。

更多 chmod 命令的使用介绍，可通查看 chmod 命令的手册页。可以得到更详尽的信息和 chmod 命令的各个参数的使用介绍。

```
chmod –help
或者使用 man 手册
man chmod
```

1.4.3 umask 设置默认权限

当创建一个文件时，umask 命令控制着文件的默认权限。umask 命令使用八进制表示法来表达从文件模式属性中删除一个位掩码。请看下面的例子：

```
bkrc@bkrc:/tmp/test$ ls
bkrc.txt
bkrc@bkrc:/tmp/test$ rm bkrc.txt
bkrc@bkrc:/tmp/test$ umask
0022
bkrc@bkrc:/tmp/test$ > bkrc.txt
bkrc@bkrc:/tmp/test$ ls -l bkrc.txt
-rw-r--r-- 1 pi pi 0 Nov 19 17:40 bkrc.txt
```

删除 bkrc.txt,让我重新开始，然后运行不带任何参数的 umask 命令，查看当前的掩码值，可以看到当前的值是 0022，这个数值是掩码的八进制表达形式。创建文件 bkrc.txt，我们可以看到文件所有者得到读权限和写权限，用户组和其他人只是得到读权限。用户组和其他人没有得到写权限的原因是由掩码值决定的。重复我们的实验，这次我们修改掩码的值。

```
bkrc@bkrc:/tmp/test$ rm bkrc.txt
bkrc@bkrc:/tmp/test$ umask 0000
```

```
bkrc@bkrc:/tmp/test$ > bkrc.txt
bkrc@bkrc:/tmp/test$ ls -l bkrc.txt
-rw-rw-rw- 1 pi pi 0 Nov 19 17:47 bkrc.txt
```

当掩码设置为 0000（实际上是关掉它）之后，可以看到用户组和其他人都可以读写文件了。

为了弄明白这是怎么回事，把掩码展开成二进制形式，然后与文件属性相比较，看看有什么区别：

|初始文件权限 | --- rw- rw- rw-|

|umask 掩码 | 000 000 000 010|

|最终文件权限 | --- rw- rw- r--|

此刻先忽略掉开头的三个零（将在下一节讲解），注意掩码中若出现一个数字 1，则删除文件模式中和这个 1 在相同位置的属性，在这是指其他人的写权限。这就是掩码要完成的任务。

掩码的二进制形式中，出现数字 1 的位置，相应地关掉一个文件模式属性。看一下掩码 0022 的作用：

|初始文件权限 | --- rw- rw- rw-|

|umask 掩码 | 000 000 010 010|

|最终文件权限 | --- rw- r- r--|

又一次，二进制中数字 1 出现的位置，相对应的属性被删除。从八进制的表示形式来看，文件的默认权限是这样计算的，用 0666 减去 umask 掩码的值得到就是文件权限（0666-0022=0644）。

1.4.4 切换用户身份

在有些时候，我们需要切换登录用户，比如需要使用超级管理员权限去执行一些管理任务。但也有可能使用“另外一个普通用户”，如测试账号。

在 Linux 系统下，有三种切换用户身份的方法。

1. 注销系统并以其他用户身份重新登录系统。

2. 使用 su 命令。

3. 使用 sudo 命令。

跳过第一种方法，因为我们知道怎样使用它，并且它缺乏其它两种方法的方便性。

在我们自己的 shell 会话中，su 命令允许你以另外一个用户的 ID 启动一个新的 shell 会话，或者是以这个用户的身份来发布一个命令。sudo 命令允许一个管理员设置一个叫做/etc/sudoers 的配置文件，并且定义了一些具体命令，在假定的身份下，特殊用户可以执行这些命令。

先介绍 su 命令。su 命令用来以另一个用户的身份来启动 shell。这个命令语法看起来像这样：

```
su [-[l]] [user]
```

如果包含"-l"选项，那么会为指定用户启动一个需要登录的 shell。这意味着会加载此用户的 shell 环境，并且工作目录会更改到这个用户的家目录。这通常是我们所需要的。如果不指定用户，那么就假定是超级用户。注意选项"-l"可以缩写为"-"，这是经常用到的形式。

启动超级用户的 shell，我们可以这样做：

```
bkrc@bkrc:/tmp/test$ su –
Password:
root@bkrc:~#
```

按下回车符之后，shell 提示我们输入超级用户的密码。如果密码输入正确，出现一个新的 shell 提示符，这表明这个 shell 具有超级用户特权（提示符的末尾字符是"#"而不是"$"），并且当前工作目录是超级用户的家目录（通常是/root）。一旦进入一个新的 shell，我们能执行超级用户所使用的命令。当工作完成后，输入"exit"，则返回到原来的 shell:

```
root@bkrc:~# exit
bkrc@bkrc:/tmp/test$
```

以这样的方式使用 su 命令，也可以只执行单个命令，而不是启动一个新的可交互的 shell：

```
su -c ‘command’
```

使用这种模式，命令传递到一个新 shell 中执行。把命令用单引号引起来很重要，因为我们不想命令在当前 shell 中展开，但需要在新 shell 中展开。

```
bkrc@bkrc:/tmp/test$ su -c ‘ls -l /root/’
Password:
一些 ls 命令的输出
bkrc@bkrc:/tmp/test$
```

sudo 以另一个用户身份执行命令，sudo 命令在很多方面都相似于 su 命令，但是

sudo 还有一些非常重要的功能。管理员能够配置 sudo 命令，从而允许一个普通用户以不同的身份（通常是超级用户），通过一种非常可控的方式来执行命令。尤其是，只有一个用户可以执行一个或多个特殊命令时，（更体现了 sudo 命令的方便性）。

另一个重要差异是 sudo 命令不要求超级用户的密码。使用 sudo 命令时，使用普通用户自己的密码来认证。例如，sudo 命令经过配置，允许我们运行一个虚构的备份程序，叫做"backup_script"，这个程序要求超级用户权限。通过 sudo 命令，这个程序会像这样运行：

```
sudo backup_script
Password:
认证完成后系统备份脚本开始备份工作
```

按下回车键之后，shell 提示我们输入我们的密码（不是超级用户的）。一旦认证完成，则执行具体的命令。su 和 sudo 之间的一个重要区别是 sudo 不会重新启动一个 shell，也不会加载另一个用户的 shell 运行环境。这意味者命令不必用单引号引起来。

想知道 sudo 命令可以授予那些权限，使用“-l”选项，可列出所有权限：

```
sudo -l
```

更多详细信息，阅读 sudo 手册页。

```
man sudo
```

1.4.5 chown 更改文件所有者和用户组

chown 命令被用来更改文件或目录的所有者和用户组。使用这个命令需要超级用户权限。chown 命令的语法看起来像这样：

```
chown [owner][:[group]] file...
```

chown 可以根据这个命令的第一个参数更改文件或目录的所有者和用户组。表 2.4.6 是一些例子：

表 2.4.6 chown 参数实例

参数	结果
bob	把文件所有者从当前属主更改为用户 bob。
bob:users	把文件所有者改为用户 bob，文件用户组改为用户组 users。
:admins	把文件用户组改为组 admins，文件所有者不变。

bob:	文件所有者改为用户 bob，文件用户组改为用户 bob 登录系统时所属的用户组。

在旧版类 Unix 系统中，chown 命令只能更改文件所有权，而不是用户组所有权。为了达到目的，使用一个独立的命令，chgrp 来完成。除了限制多一点之外，chgrp 命令与 chown 命令使用起来很相似。

1.4.6 更改用户密码

在类 Uinx 系统中使用命令 passwd 来更改你的密码（和其他用户的密码，如果你具有超级用户权限的话）。使用 passwd 命令，来设置或更改用户密码。命令语法如下所示：

```
passwd [user]
```

只要输入 passwd 命令，就能更改你的密码。shell 会提示你输入你的旧密码和你的新密码：

```
passwd
(current) UNIX password:
New UNIX password:
```

passwd 命令将会试着强迫你使用“强”密码。这意味着它会拒绝接受太短的密码、与先前相似的密码、字典中的单词作为密码或者是太容易猜到的密码。

如果是超级用户权限，则可以指定一个用户名作为 passwd 命令的参数，这样可以设置另一个用户的密码。还有其它的 passwd 命令选项对超级用户有效，允许帐号锁定，密码失效，等等。详细内容，参考 passwd 命令的手册页。

```
man passwd
```

1.5 shell 环境

1.5.1 环境变量储存内容

printenv -- 打印部分或所有的环境变量

set -- 设置 shell 选项

export — 导出环境变量，让随后执行的程序知道。

alias -- 创建命令别名

在 shell 会话中保存着大量信息。这些信息被称为(shell 的)环境。

程序获取环境中的数据（即环境变量）来了解本机的配置。虽然大多数程序用配置文件来存储程序设置，但一些程序会根据环境变量来调整他们的行为。

shell 在环境中存储了两种基本类型的数据，虽然 bash 几乎无法分辨这些数据的类型。它们是环境变量和 shell 变量。Shell 变量是 bash 存放的少量数据。剩下的都是环境变量。除了变量，shell 也存储了一些可编程的数据，即命令别称和 shell 函数。

1.5.2 Shell 环境变量

我们可以用 bash 的内建命令 set，或者是 printenv 程序来查看环境变量。set 命令可以显示 shell 或环境变量，而 printenv 只是显示环境变量。因为环境变量列表比较长，最好把每个命令的输出通过管道传递给 less 来阅读：

```
bkrc@bkrc:~$ printenv | less
执行完上面的命令后，我们应该能看到如下输出：
SHELL=/bin/bash
COLORTERM=truecolor
XDG_CONFIG_DIRS=/etc/xdg
XDG_MENU_PREFIX=lxde-pi-
HOSTNAME=raspberrypi
LANGUAGE=en_US.UTF-8
_LXSESSION_PID=721
SSH_AUTH_SOCK=/tmp/ssh-1fi7jpDtO2l1/agent.721
XDG_CONFIG_HOME=/home/pi/.config
DESKTOP_SESSION=LXDE-pi
```

我们所看到的是环境变量及其数值的列表，比如，有一个 USER 变量，这个变量的值为 pi，可以通过下面命令来查看特定变量的值

```
bkrc@bkrc:~$ printenv USER
bkrc
```

当使用没有带选项和参数的 set 命令时，shell 变量，环境变量，和定义的 shell 函数都会被显示。不同于 printenv 命令，set 命令的输出很友好地按照首字母顺序排列：

```
bkrc@bkrc:~$ set |less
```

也可以通过 echo 命令来查看一个变量的内容，就像这样：

```
bkrc@bkrc:~$ echo $HOME
/home/bkrc
```

别名无法通过使用 set 或 printenv 来查看。

用不带参数的 alias 来查看别名:

```
bkrc@bkrc:~$ alias
alias egrep='egrep --color=auto'
alias fgrep='fgrep --color=auto'
alias grep='grep --color=auto'
alias ls='ls --color=auto'
```

1.5.3 特殊的环境变量

shell 环境中包含相当多的变量，表 1.5.1 列举一些特殊的环境变量

表 1.5.1 环境变量

DISPLAY	如果你正在运行图形界面环境，那么这个变量就是你显示器的名字。通常，它是 ":0"，意思是由 X 产生的第一个显示器。
SHELL	shell 程序的名字。
HOME	用户家目录
LANG	定义了字符集以及语言编码方式。
PATH	由冒号分开的目录列表，当你输入可执行程序名后，会搜索这个目录列表。
PS1	Prompt String 1. 这个定义了你的 shell 提示符的内容。随后我们可以看到，这个变量内容可以全面地定制。

1.5.4 Shell 环境配置文件

当我们登录系统后，bash 程序启动，并且会读取一系列称为启动文件的配置脚本，这些文件定义了默认的可供所有用户共享的 shell 环境。然后是读取更多位于我们自己家目录中的启动文件，这些启动文件定义了用户个人的 shell 环境。确切的启动顺序依赖于要运行的 shell 会话类型。有两种 shell 会话类型：一个是登录 shell 会话，另一个是非登录 shell 会话。

登录 shell 会话会在其中提示用户输入用户名和密码；例如，我们启动一个虚拟控制台会话。非登录 shell 会话是指在 GUI 下启动的终端会话。

登录 shell 会读取一个或多个启动文件。如表 1.5.2 所示：

表 1.5.2 登陆 shell 会话的启动文件

文件	内容
/etc/profile	应用于所有用户的全局配置脚本。
~/.bash_profile	用户个人的启动文件。可以用来扩展或重写全局配置脚本中的设置。
~/.bash_login	如果文件 ~/.bash_profile 没有找到，bash 会尝试读取这个脚本。
~/.profile	如果文件 ~/.bash_profile 或文件 ~/.bash_login 都没有找到，bash 会试图读取这个文件。 这是基于 Debian 发行版的默认设置，比方说 Ubuntu。

非登录 shell 会话会读取以下启动文件：如表 1.5.3 所示

表 1.5.3 非登陆 shell 会话启动文件

文件	内容
/etc/bash.bashrc	应用于所有用户的全局配置文件。
~/.bashrc	用户个人的启动文件。可以用来扩展或重写全局配置脚本中的设置。

除了读取以上启动文件之外，非登录 shell 会话也会继承它们父进程的环境设置，通常是一个登录 shell。浏览一下你的文件系统，看一看系统中有哪些启动文件。上面列出的大多数文件名都以圆点开头（意味着它们是隐藏文件），你需要使用带"-a"选项的 ls 命令。

对于普通用户来说，文件~/.bashrc 可能是最重要的启动文件，因为它几乎总是被读取。非登录 shell 默认会读取它，并且大多数登录 shell 也会最终读取它。

1.5.5 修改 Shell 环境

export 命令告诉 shell 让 shell 的子进程使用 export 定义的变量内容。

```
export PATH=$PATH:~/bin
```

如果未将 export 命令写入启动配置脚本中，那 export 定义的变量仅对当前 shell 有效，退出当前 shell，将丢失。如果需要持久化，则应该将 shell 变量定义写入启动配置脚本中，通常是~/.bashrc 文件，在修改 shell 环境时，最好谨慎行事。

```
echo “export PATH=$PATH:~/bin” >> ~/.bashrc
```

1.6 文本编辑器

1.6.1 vim 的使用

Vim 是一个可以在 shell 中运行的高度可自定义的文本编辑器。Bram Moolenaar 因其在 1988 年开始的 Vim 的发展而受到赞誉。

vim 简洁而强大，上手比较困难。下面将 vim 进行一些外观优化

```
$ sudo apt-get install vim
$ wget --no-check-certificate
https://raw.githubusercontent.com/amix/vimrc/master/vimrcs/basic.vim
//  复制 vim 的配置文件到你的~/.vimrc
$ cp basic.vim ~/.vimrc
```

优化后编辑 Python 文件时将会高亮显示语法关键字。

基本上 vi/vim 共分为三种模式，分别是命令模式（Command mode），输入模式（Insert mode）和底线命令模式（Last line mode）。

命令模式：用户刚刚启动 vi/vim，便进入了命令模式。

此状态下敲击键盘动作会被 Vim 识别为命令，而非输入字符。比如我们此时按下 i，并不会输入一个字符，i 被当作了一个命令。

以下是常用的几个命令：

i 切换到输入模式，以输入字符。

x 删除当前光标所在处的字符。

: 切换到底线命令模式，以在最底一行输入命令。

若想要编辑文本：启动 vim，进入了命令模式，按下 i，切换到输入模式。

命令模式只有一些最基本的命令，因此仍要依靠底线命令模式输入更多命令。

输入模式，在命令模式下按下 i 就进入了输入模式。在输入模式中，可以使用以下按键：

ENTER，回车键，换行

BACK SPACE，退格键，删除光标前一个字符

DEL，删除键，删除光标后一个字符

方向键，在文本中移动光标

HOME/END，移动光标到行首/行尾

Page Up/Page Down，上/下翻页

Insert，切换光标为输入/替换模式，光标将变成竖线/下划线

ESC，退出输入模式，切换到命令模式

底线命令模式，在命令模式下按下:（英文冒号）就进入了底线命令模式。底线命令模式可以输入单个或多个字符的命令，可用的命令非常多。在底线命令模式中，基本的命令有（已经省略了冒号）。

q 退出程序

w 保存文件

按 ESC 键可随时退出底线命令模式。

要熟练使用 vim，则需要参考图 1.6.1 勤加练习

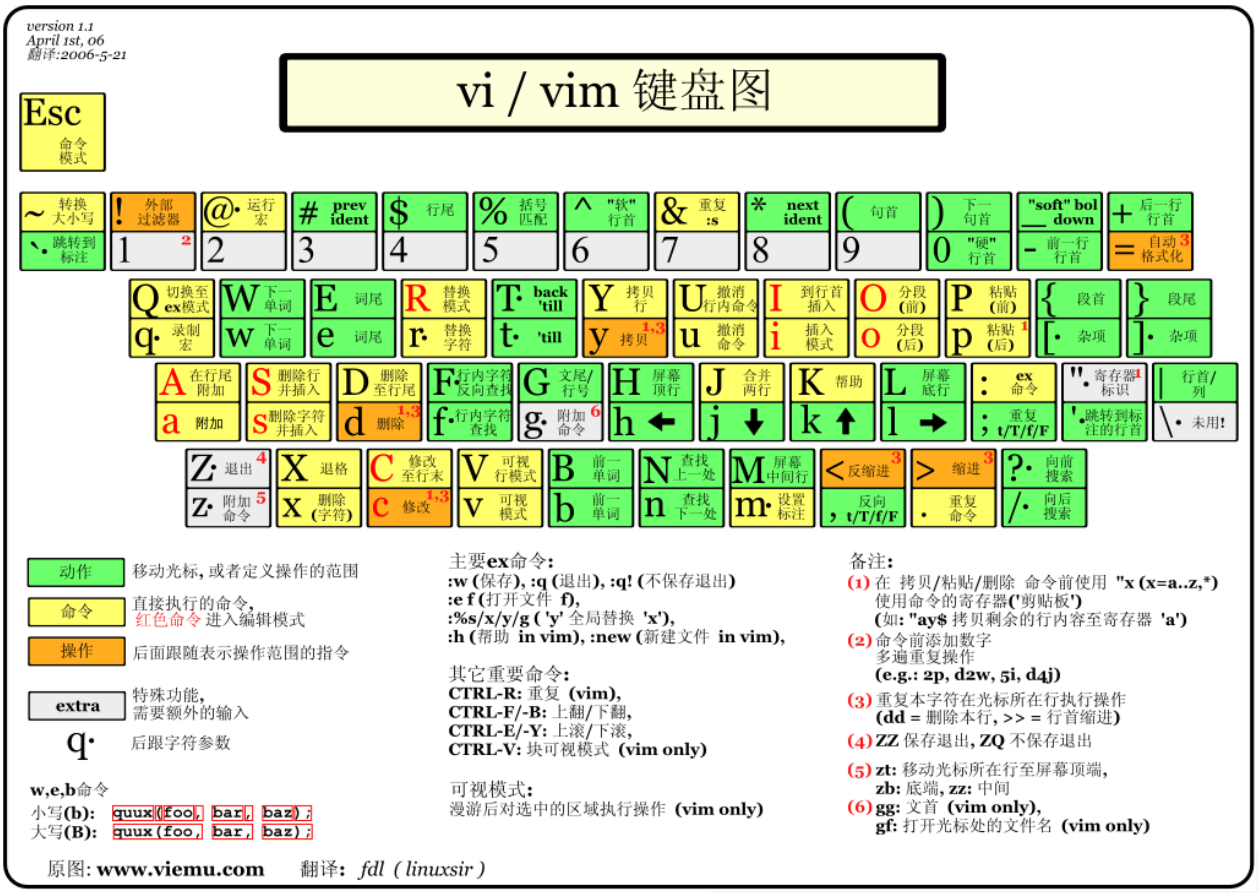

图 1.6.1 vim 键盘图

1.6.2 其他文本编辑器

所有的文本编辑器都可以通过在命令行中输入编辑器的名字，加上你所想要编辑的文件来唤醒。如果所输入的文件名不存在，编辑器则会假定你想要创建一个新文件。下面是一个使用 gedit 的例子：

```
gedit some_file_name
```

这条命令将会启动 gedit 文本编辑器，同时加载名为"some_file_name"的文件，如果这个文件存在的话。

所有的图形文本编辑器很大程度上都是不需要解释的，所以在这里不会介绍它们。反之，将集中精力介绍基于文本的文本编辑器，nano。

让我们启动 nano，并且编辑文件~/.bashrc。但是当我们编辑一个重要的配置文件时，首先创建一个这个配置文件的备份，以防万一。创建文件~/ .bashrc 的备份文件命令如下：

```
bkrc@bkrc:~$ cp ~/.bashrc bashrc.back
```

备份文件的名字无关紧要，只要选择一个容易理解的文件名。扩展名".bak"、".sav"、".old"和 ".orig"都是用来标识备份文件的流行方法。记住 cp 命令会默默地覆盖已经存在的同名文件。

有了备份文件，开始启动 nano 编辑器：

```
bkrc@bkrc:~$ nano .bashrc
```

一但启动 nano 编辑器，你将得到想下面一样的文字

```
GNU nano 3.2                                .bashrc
…
```

使用 nano 编辑器，你应该学习的第一个命令是如何退出。你输入 Ctrl-x 来退出 nano。在屏幕底层的菜单中说明了这个命令。"^X" 表示法意思是 Ctrl-x。这是控制字符的常见表示法，许多程序都使用它。

第二个命令是学习如何保存修改后的文件。。对于 nano 来说是 Ctrl-o。

尝试练习使用 nano，如写入一些配置信息，具体解释见表 2.6.1

```
# Change umask to make directory sharing easier
umask 0002
 # Ignore duplicates in command history and increase
 # history size to 1000 lines
export HISTCONTROL=ignoredups
export HISTSIZE=1000
 # Add some helpful aliases
alias l.='ls -d .* --color=auto'
alias ll='ls -l --color=auto'
```

当我们完成修改后，输入 Ctrl-o 来保存我们修改的.bashrc 文件，输入 Ctrl-x 退

出 nano。

表 1.6.1 说明

文本行	含义
umask 0002	设置掩码来解决共享目录的问题。
export HISTCONTROL=ignoredups	使得 shell 的历史记录功能忽略一个命令，如果相同的命令已被记录。
export HISTSIZE=1000	增加命令历史的大小，从默认的 500 行扩大到 1000 行。
alias l.='ls -d .* --color=auto'	创建一个新命令，叫做'l.'，这个命令会显示所有以点开头的目录项。
alias ll='ls -l --color=auto'	创建一个叫做'll'的命令，这个命令会显示长格式目录列表。

1.7 存储媒介

1.7.1 fstab 文件字段

储存设备需要事先挂载到文件系统树中，才能被使用。在类 Uninx 系统中有一个叫做/etc/fstab 的文件可以列出系统启动时要挂载的设备（磁盘分区），可以看看这个文件内容，如下：

```
bkrc@bkrc:~$ cat /etc/fstab
```

```
proc                    /proc           proc    defaults            0       0
PARTUUID=5e3da3da-01    /boot           vfat    defaults            0       2
PARTUUID=5e3da3da-02    /               ext4    defaults,noatime    0       1
# a swapfile is not a swap partition, no line here
#     use     dphys-swapfile swap[on|off]     for that
```

在上面实例中所列出的大多数文件系统是虚拟的，现在并不适合讨论。只有一个是我们感兴趣的

```
PARTUUID=5e3da3da-02    /               ext4    defaults,noatime    0       1
```

每一行都是硬盘分区信息，每行由六个字段组成，如表 1.7.1 所示

表 1.7.1 /etc/fstab 字段

字段	内容	说明
1	设备名	传统上，这个字段包含与物理设备相关联的设备文件的实际名字，比如说/dev/hda1（第一个 IDE 通道上第一个主设备分区）。然而今天的计算机，有很多热插拔设备（像 USB 驱动设备），许多 现代的 Linux 发行版用一个文本标签和设备相关联。当这个设备连接到系统中时， 这个标签（当储存媒介格式化时，这个标签会被添加到存储媒介中）会被操作系统读取。那样的话，不管赋给实际物理设备哪个设备文件，这个设备仍然能被系统正确地识别。
2	挂载点	设备所连接到的文件系统树的目录。
3	文件系统类型	Linux 允许挂载许多文件系统类型。大多数本地的 Linux 文件系统是 ext3，但是也支持很多其它的，比方说 FAT16(msdos),FAT32(vfat)，NTFS(ntfs)，CD-ROM (iso9660)，等等。

4	选项	文件系统可以通过各种各样的选项来挂载。有可能，例如，挂载只读的文件系统， 或者挂载阻止执行任何程序的文件系统（一个有用的安全特性，避免删除媒介。）
5	频率	一位数字，指定是否和在什么时间用 dump 命令来备份一个文件系统。
6	次序	一位数字，指定 fsck 命令按照什么次序来检查文件系统。

1.7.2 挂载和卸载储存设备

命令 mount 用来挂载文件系统，执行不带参数的 mount 命令，将会显示当前挂载的文件系统：

```
bkrc@bkrc:~$ mount
/dev/mmcblk0p2 on / type ext4 (rw,noatime)
devtmpfs on /dev type devtmpfs (rw,relatime,size=1841404k,nr_inodes=117887,mode=755)
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
proc on /proc type proc (rw,relatime)
securityfs on /sys/kernel/security type securityfs (rw,nosuid,nodev,noexec,relatime)
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev)
…
```

这个列表的格式是：设备 on 挂载点 type 文件系统类型（选项）。例如，第一行所示设备/dev/mmcblkop2 作为根文件系统被挂载，文件系统类型是 ext4，并且可读可写（“rw”选项）。

使用 mount 命令挂载储存设备，命令如下：

```
sudo mount /dev/sdb1 /mnt
```

卸载储存设备使用 umount 命令，需要注意的是，不能在挂载目录下卸载。

```
sudo /dev/sdb1
```

或者

```
sudo umount /mnt
```

挂载一个储存设备的关键是如何确定储存设备的名称，首先，让我们看一下系统怎样来命名设备。如果我们列出目录/dev（所有设备的住所）的内容，我们会看到许许多多的设备：

```
ls /dev
argon-h264mem      mem                    shm      tty39      vcio
argon-hevcmem      memory_bandwidth       snd      tty4       vc-mem
```

表 1.7.2 罗列了一些储存设备命名模式

表 1.7.2 Linux 储存设备名称

模式	设备
/dev/fd*	软盘驱动器
/dev/hd*	老系统中的 IDE(PATA)磁盘。典型的主板包含两个 IDE 连接器或者是通道，每个连接器带有一根缆线，每根缆线上有两个硬盘驱动器连接点。缆线上的第一个驱动器叫做主设备， 第二个叫做从设备。设备名称这样安排，/dev/hda 是指第一通道上的主设备名；/dev/hdb 是第一通道上的从设备名；/dev/hdc 是第二通道上的主设备名，等等。末尾的数字表示硬盘驱动器上的分区。例如，/dev/hda1 是指系统中第一硬盘驱动器上的第一个分区，而/dev/hda 则是指整个硬盘驱动器。
/dev/lp*	打印机
/dev/sd*	SCSI 磁盘。在最近的 Linux 系统中，内核把所有类似于磁盘的设备（包括 PATA/SATA 硬盘，闪存，和 USB 存储设备，比如说可移动的音乐播放器和数码相机）看作 SCSI 磁盘。剩下的命名系统类似于上述所描述的旧的/dev/hd*命名方案。
/dev/sr*	光盘（CD/DVD 读取器和烧写器）

使用下面技巧，可以很方便的查看到当有移动设备连接后，它的设备名称。执行如下命令，然后插入 U 盘。

```
sudo tail -f /var/log/messages
```

这个文件的最后几行将会被显示，然后停止，当插入移动设备后，内核就会发现这个设备，并且将日志写入文件/var/log/messages 中。

输出结果包含该移动设备的设备名，你应该能看到这两行，输入 Ctrl-c，重新得到提示符。

```
Nov 20 15:37:28 raspberrypi kernel: [ 4830.670717]    sda: sda1
Nov 20 15:37:28 raspberrypi kernel: [ 4830.674297] sd 0:0:0:0: [sda] Attached SCSI
removable disk
```

这告诉我们这个设备名称是/dev/sdb 指整个设备，/dev/sdb1 是这个设备的第一分区。这个设备名称会保持不变只要设备与计算机保持连接并且计算机不会重新启动。

1.7.3 数据移入和移出

使用 dd 命令可以将数据块从一个地方复制到另一个地方，由于历史的原因，它有独特的语法，我们经常这样的使用它。

```
dd if=input_file of=output_file [bs=block_size [count=blocks]]
```

比方说我们有两个相同容量的 USB 闪存驱动器，并且要精确地把第一个驱动器（中的内容）复制给第二个。如果连接两个设备到计算机上，它们各自被分配到设备/dev/sdb 和/dev/sdc 上，这样我们就能通过下面的命令把第一个驱动器中的所有数据复制到第二个驱动器中。

```
dd if=/dev/sdb of=/dev/sdc
```

或者，如果只有第一个驱动器被连接到计算机上，我们可以把它的内容复制到一个普通文件中供以后恢复或复制数据：

```
dd if=/dev/sdb of=flash_drive.img
```

警告！dd 命令非常强大，if 指定输入源，of 指定输出源，如果我们把 if 和 of 搞反了，那 dd 命令就不是在做磁盘备份了，而是清除磁盘。在按下回车之前，要再三检查输入与输出规范。

1.8 网络相关

1.8.1 检查和检测网络

ping 是最基本的网络命令，常用来检测一个网络的性能和运作情况。ping 命令发送一个特殊的网络数据包，叫做 ICMP ECHO_REQUEST 到一台指定的主机。大多数接收这个包的网络设备将会回复它，来允许网络连接验证。

例如，看看我们能否连接到网站 r8c.com(这是百科荣创的官方网站)，可以这样使用 ping 命令：

```
ping r8c.com
```

ping 命令启动后会持续在特定的时间间隔内（默认是一秒）发送数据包，直到它被中断。

```
64 bytes from 123.56.250.111: icmp_seq=1 ttl=50 time=26.2 ms
64 bytes from 123.56.250.111: icmp_seq=2 ttl=50 time=25.0 ms
64 bytes from 123.56.250.111: icmp_seq=3 ttl=50 time=33.3 ms
64 bytes from 123.56.250.111: icmp_seq=4 ttl=50 time=43.3 ms
```

按下组合键 Ctrl-c，中断这个命令之后，ping 打印出运行统计信息。一个正常工作的网络会报告零个数据包丢失。一个成功执行的“ping”命令会意味着网络的各个部件（网卡，电缆，路由，网关）都处于正常的工作状态。

使用命令 traceroute（在一些系统中使用 tracepath 代替）来查看从本地到指定主机要经过哪些路由器的转发，例如，看一下到达 r8c.com 需要经过的路由器，命令如下：

```
tracearroute r8c.com
```

对于那些提供标识信息的路由器，我们能看到它们的主机名，IP 地址和网络延时，这些数据包括三次从本地到此路由器的往返时间样本。对于那些没有提供标识信息的路由器（由于路由器配置，网络拥塞，防火墙等方面的原因），我们会看到三个星号。

netstat 命令用来检查各种各样的网络设置和统计数据。通过此命令的许多参数选项，可以查看网络设置中的各种特性。使用“-ie”选项，能够查看系统中的网络接口：

```
bkrc@bkrc:~$ netstat -ie
Kernel Interface table
eth0: flags=4099<UP,BROADCAST,MULTICAST>    mtu 1500
        ether dc:a6:32:24:71:31    txqueuelen 1000    (Ethernet)
        RX packets 0    bytes 0 (0.0 B)
        RX errors 0    dropped 0    overruns 0    frame 0
        TX packets 0    bytes 0 (0.0 B)
```

```
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 427  bytes 32330 (31.5 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 427  bytes 32330 (31.5 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.12.1  netmask 255.255.255.0  broadcast 192.168.12.255
        inet6 fe80::8cc7:a153:40b0:9353  prefixlen 64  scopeid 0x20<link>
        ether dc:a6:32:24:71:32  txqueuelen 1000  (Ethernet)
        RX packets 621  bytes 87624 (85.5 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 760  bytes 732716 (715.5 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

在上面的测试系统中有三个网络接口。第一个，叫做 eth0，是以太网接口，和第二个，叫做 lo，是内部回环网络接口，它是一个虚拟接口，系统用它来“自言自语”。第三个，叫做 wlan0，是无线网接口。

当执行日常网络诊断时，要查看的重要信息是每个网络接口第一行开头的单词“UP”，说明这个网络接口已经生效，还要查看第二行中 inet addr 字段出现的有效 IP 地址。对于使用 DHCP（动态主机配置协议）的系统，在这个字段中的一个有效 IP 地址则证明了 DHCP 工作正常。

使用-r 选项显示内核的网络路由表

```
bkrc@bkrc:~$ netstat -r
Kernel IP routing table
Destination     Gateway         Genmask         Flags   MSS Window  irtt Iface
default         0.0.0.0         0.0.0.0         U       0 0         0 wlan0
link-local      0.0.0.0         255.255.0.0     U       0 0         0 wlan0
192.168.12.0    0.0.0.0         255.255.255.0   U       0 0         0 wlan0
```

第一行包含目的地 default。指的是发往任何表上没有列出的目的地网络的流量。在路由表里面没有匹配的项的数据发到默认路由去。网关为 0.0.0.0 的网络通常是直连到网络设备上的。因为到自己的直连设备上是不需要网关的，0.0.0.0 的路由是你的缺

省网关。

这是位于防火墙之内的局域网中，一台客户端计算机的典型路由表。第三行显示了目的地 192.168.12.0。IP 地址以零结尾是指网络，而不是独立主机，所以这个目的地指的是局域网中的任何一台主机。下一个字段 Gateway，是网关（路由器）的名字或 IP 地址，用它来连接当前的主机和目的地的网络。若这个字段显示一个星号，则表明不需要网关。

更多关于 netstat 命令的详细使用，请查看 netstat 命令的手册

```
man netstat
```

1.8.2 与远程主机安全通信

通过网络来远程操控类 Unix 的操作系统已经有很多年的历史。早些年，在因特网普遍推广之前，人们使用 rlogin 和 telnet 程序来登录远程主机。然而这些程序，拥有和 ftp 程序一样的致命缺点；它们以明码形式来传输所有的交流信息（包括登录命令和密码）。由于明码传输导致安全性极低。为了解决这个问题，开发了一款新的协议，叫做 SSH（Secure Shell）。

SSH 解决了这两个基本的和远端主机安全交流的问题。首先，它要认证远端主机是否为它所知道的那台主机（这样就有效阻止所谓的“中间人”的攻击），其次，它加密了本地与远程主机之间所有的通讯信息。

SSH 由两部分组成。SSH 服务端运行在远端主机上，在端口 22 上监听收到的外部连接，而 SSH 客户端用在本地系统中，用来和远端服务器通信。

用来与远端 SSH 服务器相连接的 SSH 客户端程序，顺理成章，叫做 ssh。使用 localhost 作为远端主机的名字。这种情况下，计算机会和它自己创建网络连接。

```
bkrc@bkrc:~$ ssh localhost
The authenticity of host 'localhost (::1)' can't be established.
ECDSA key fingerprint is SHA256:N/EcI209DYNFQdgkcT/8s8QeFh6LEwX5dgu8vQyFU7c.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added 'localhost' (ECDSA) to the list of known hosts.
pi@localhost's password:
Linux raspberrypi 4.19.75-v7l+ #1270 SMP Tue Sep 24 18:51:41 BST 2019 armv7l
The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
```

```
bkrc@bkrc:~$
```

第一次尝试连接，提示信息表明远端主机的真实性不能确立。这是因为客户端程序以前从没有看到过这个远端主机。为了接受远端主机的身份验证凭据，输入“yes”。一旦建立了连接，会提示用户输入远端主机的登录密码：

```
Warning: Permanently added 'localhost' (ECDSA) to the list of known hosts.
pi@localhost's password:
```

成功地输入密码之后，我们会接收到远端系统的 shell 提示符：

```
bkrc@bkrc:~$
```

远端 shell 会话一直存在，直到用户输入 exit 命令后，则关闭了远程连接。这时候，本地的 shell 会话恢复，本地 shell 提示符重新出现。

当我们知道远程主机的 IP 地址，远程主机 ssh 登录端口号和登录用户名时，这里假设远程主机的 IP 地址为 192.168.12.1，ssh 协议端口号为默认 22。登录用户名为 bkrc，可以使用如下命令进行登入：

```
ssh bkrc@192.168.12.1 -p 22
```

如果是第一次连接，需要输入 yes 来接收远端主机的身份凭证，之后输入远端主机的登入密码。

在 Window 机器中，使用 ssh 协议远程登入 Linux 服务器，来完成一些实际的工作，如果你使用的是 Win10，可直接打开 PowerShell，输入如下命令来登录：

```
ssh userName@remoteIP -p 22
```

或者找一个 Windows 平台下的 SSH 客户端，有很多这样的工具，如 putty、Winscp 等等。再提供的资料包中已经包含了这一些软件，安装即可使用。

1.8.3 SSH 运行 GUI 程序配置

在默认情况下，ssh 远程登陆的 shell 会话，可以执行绝大多数程序，但无法执行带 GUI 界面的程序，这非常的不合理，可通过下面的设置来解除这一限制。修改 ssh 配置，打开 ssh 转发 X11 的选项。

```
sudo vim /etc/ssh/sshd_config
```

找到这几行，将#号去掉

```
UsePam yes
X11Forwarding yes
X11DisplayOffset 10
X11UseLocalhost yes
```

```
PermitTTY yes
PrintMotd no
```

保存修改，然后重启 SSH 服务

```
bkrc@bkrc:~$ sudo systemctl restart sshd.service
```

在 ssh 客户端中重新登陆远端主机，在远端 shell 会话中运行带 GUI 界面程序是，设置一些 DISPLAY 这个环境变量，来指定显示那个具体的屏幕

```
bkrc@bkrc:~$ export DISPLAY=0.0
```

如果需要环境变量持久有效，则写入启动配置文件中。

```
bkrc@bkrc:~$ echo “export DISPLAY=0.0” >> ~/.bashrc
```

经过这样的配置后，你便可以在远端 shell 会话中启动 GUI 程序，注意 GUI 界面将显示在板子所连接的屏幕上，而非显示电脑屏幕上。

有些时候我们希望，远端 shell 会话启动的 GUI 程序的界面能显示在本地电脑屏幕上，以便开发调试，实际上 VNC 可以完成这需求，当 VNC 网络开销比较大。一个更好的替代方案是使用 putty 和 xming 工具。Putty 是一个轻量级的 ssh 客户端，xming 是一个 Windows 平台的桌面服务程序，用来显示 ssh 转发过来的 X11 界面。

先从配套资料中的常用工具中找到并安装这两个工具。首先启动 xming 程序，默认情况下 xming 将会开启一个 Xming Server0.0 的桌面服务。记住这个。

打开 putty 工具，新建一个连接，在填入一些基本的会话信息后，点击保存。

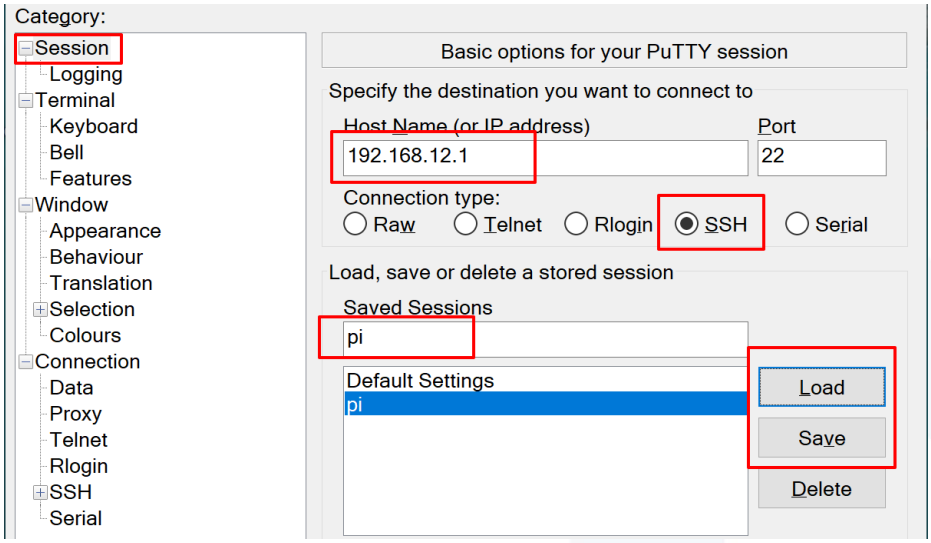

图 1.8.1 putty 填写登录信息

然后点击 SSH,再点击其下的子菜单 X11，在 x display location 字段中填入 localhost:0.0,如图 1.8.2 配置 DISPLAY 环境变量所示

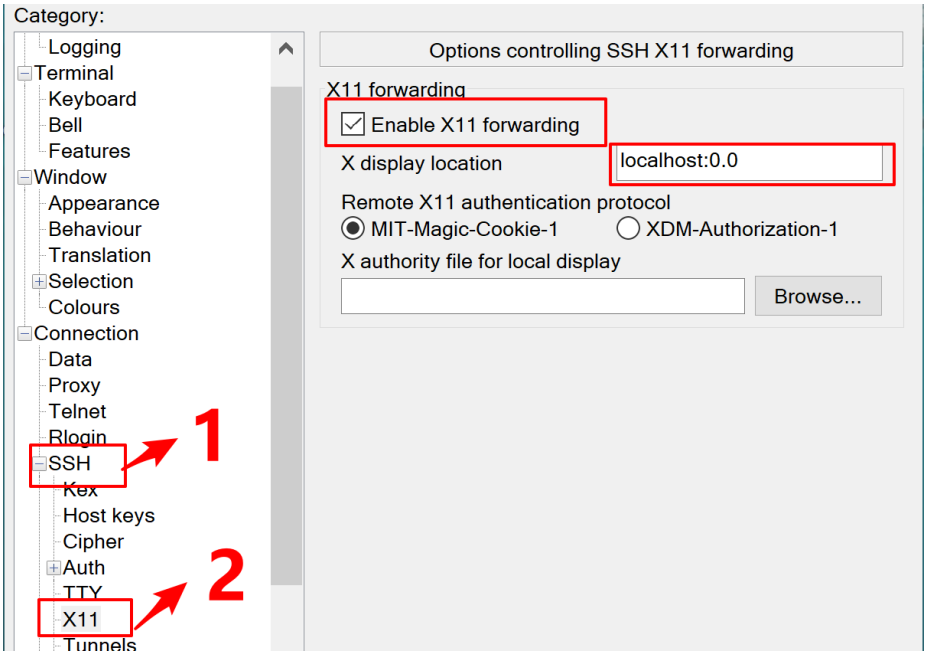

图 1.8.2 配置 DISPLAY 环境变量

最后回到 Session 选项中点击 Save。然后点击 Open 来打开会话。

准确输入密码后，将会得到远端 shell 会话。键入如下命令来查看 DISPLAY 环境变量是不是正确指向到本地的 xming 桌面服务中来。

```
bkrc@bkrc:~$ echo $DISPLAY
localhost:10.0
```

如果 DISPLAY 的值为 localhost:xx.xx 的话，表明设置成功。

最后让我们启动一个 GUI 程序来验证一下。使用如下命令来启动浏览器

```
bkrc@bkrc:~$ x-www-browser
```

一旦命令启动成功，稍等几秒钟就能在电脑屏幕中看到浏览器，如图 1.8.3 所示

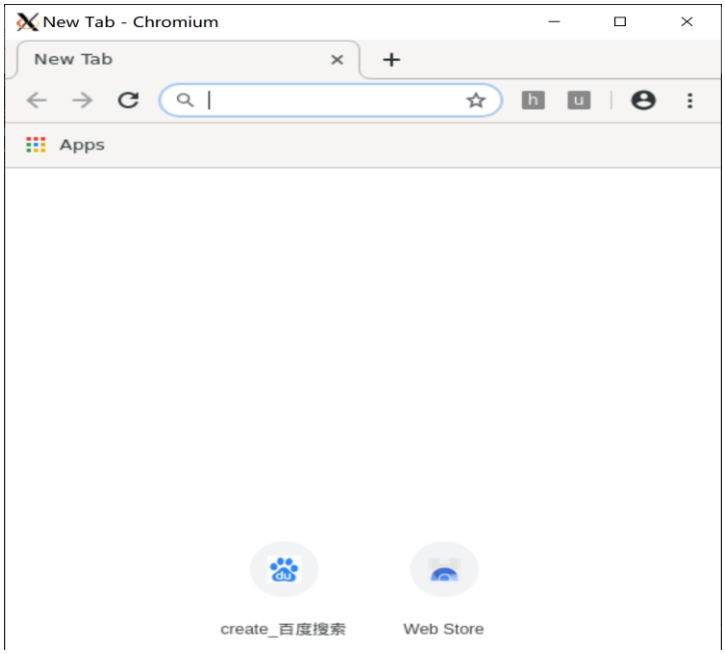

图 1.8.3 chromium 浏览器

按 Ctrl + C 来关闭浏览器，至此，电脑远程运行 GUI 界面程序配置教程结束。这两种使用方法在后续程序开发调试阶段非常实用，希望独家牢记。

1.9 文件压缩与归档

1.9.1 解压缩文件

纵观计算领域的发展史，人们努力想把最多的数据存放到到最小的可用空间中，不管是内存、存储设备或者是网络带宽。今天我们能在互联网上观看高清的网络电视都应归功于高效的数据压缩技术。数据压缩是一个删除冗余数据的过程。比方说我们有一张100*100*1 像素的纯黑的图像。根据数据存储方案（假定每个像素占 24 位，或者 3 个字节），那么这张图像将会占用 30,000 个字节的存储空间。一张单色图像包含的数据全是多余的。聪明的人们，用这种方法来编码这些数据，简单地描述这个事实，我们有 3 万个黑色的像素数据块。所以，我们不存储包含 3 万个 0（通常在图像文件中，黑色由 0 来表示）的数据块，取而代之，我们把这些数据压缩为数字 30,000，后跟一个 0，来表示我们的数据。这种数据压缩方案被称为游程编码，是一种最基本的压缩技术。今天的技术更加先进和复杂，但是基本目标依然不变——避免多余数据。

压缩算法可分为两大类，无损压缩和有损压缩。无损压缩保留了原始文件的所有数据。这意味着，当还原一个压缩文件的时候，还原的文件与原文件一模一样。

而有损压缩，执行压缩操作时会删除数据，允许更大的压缩率。当一个有损文件被还原时，它与原文件不相匹配;相反，它是一个近似值。有损压缩的例子有 JPEG（图像）文件和 MP3（音频）文件。这里将讨论无损压缩，因为计算机中的大多数数据是不能容忍丢失任何数据的。

gzip 用来压缩一个或多个文件。当执行 gzip 命令时，则原始文件的压缩版会替代原始文件。相对应的 gunzip 程序被用来把压缩文件复原为没有被压缩的版本。这里例子如下：

```
$ ls -l /etc > foo.txt
$ ls    foo.*
foo.txt
$ gzip foo.txt
$ ls foo.*
foo.txt.gz
$ gunzip foo.txt.gz
$ ls foo.*
foo.txt
```

在上面例子中，我们创建了一个名为 foo.txt 的文本文件。接下来，我们运行 gzip

命令，它会把原始文件替换为一个叫做 foo.txt.gz 的压缩文件。在 foo.* 文件列表中，我们看到原始文件已经被压缩文件替代了，并将这个压缩文件大约是原始文件的五分之一。接下来，我们运行 gunzip 程序来解压缩文件。随后，能见到压缩文件已经被原始文件替代了。

gzip 命令有许多选项，如表 1.9.1 所示：

表 1.9.1 gzip 选项

选项	说明
-c	把输出写入到标准输出，并且保留原始文件。也有可能用--stdout 和--to-stdout 选项来指定。
-d	解压缩。正如 gunzip 命令一样。也可以用--decompress 或者--uncompress 选项来指定。
-f	强制压缩，即使原始文件的压缩文件已经存在了，也要执行。也可以用--force 选项来指定。
-h	显示用法信息。也可用--help 选项来指定。
-l	列出每个被压缩文件的压缩数据。也可用--list 选项。
-r	若命令的一个或多个参数是目录，则递归地压缩目录中的文件。也可用--recursive 选项来指定。
-t	测试压缩文件的完整性。也可用--test 选项来指定。
-v	显示压缩过程中的信息。也可用--verbose 选项来指定。
-number	设置压缩指数。number 是一个在 1（最快，最小压缩）到 9（最慢，最大压缩）之间的整数。 数值 1 和 9 也可以各自用--fast 和--best 选项来表示。默认值是整数 6。

使用举例，如果只是为了浏览一下压缩文本文件的内容，可以这样做：

```
$ gunzip -c foo.txt.gz | less
```

bzip2 程序与 gzip 程序相似，但是使用了不同的压缩算法，舍弃了压缩速度，而实现了更高的压缩级别。在大多数情况下，它的工作模式等同于 gzip。由 bzip2 压缩的文件，用扩展名 bz2 来表示：

```
$ ls -l /etc > foo.txt
$ ls foo.txt
foo.txt
$ bzip2 foo.txt
$ ls -l foo.txt.bz2
foo.txt.bz2
$ bunzip2 foo.txt.bz2
```

bzip2 程序使用起来和 gzip 程序一样。我们之前讨论的 gzip 程序的所有选项

（除了-r），bzip2 程序同样也支持。注意，然而，压缩级别选项（-number）对于 bzip2 程序来说，有少许不同的含义。伴随着 bzip2 程序，有 bunzip2 和 bzcat 程序来解压缩文件。bzip2 文件也带有 bzip2recover 程序，其会试图恢复受损的 .bz2 文件。

1.9.2 归档文件

一个常见的，与文件压缩结合一块使用的文件管理任务是归档。归档就是收集许多文件，并把它们捆绑成一个大文件的过程。归档经常作为系统备份的一部分来使用。当把旧数据从一个系统移到某种类型的长期存储设备中时，也会用到归档程序。

在类 Unix 的软件世界中，这个 tar 程序是用来归档文件的经典工具。它的名字，是 tapearchive 的简称，揭示了它的根源，它是一款制作磁带备份的工具。而它仍然被用来完成传统任务，它也同样适用于其它的存储设备。我们经常看到扩展名为 .tar 或者 .tgz 的文件，它们各自表示“普通”的 tar 包和被 gzip 程序压缩过的 tar 包。一个 tar 包可以由一组独立的文件，一个或者多个目录，或者

两者混合体组成。命令语法如下：

```
tar mode[options] pathname...
```

这里的 mode 是指以下操作模式（这里只展示了一部分，查看 tar 的手册来得到完整列表）之一：如表 1.9.2 所示

表 1.9.2 tar 模式

模式	说明
c	为文件和 / 或目录列表创建归档文件。
x	抽取归档文件。
r	追加具体的路径到归档文件的末尾。
t	列出归档文件的内容。

tar 命令的例子如下：

将几个文件夹归档到一个归档包里

```
$ ls -l ./
A    B    C     #当前路径有三个文件夹
tar cvf ABC.tar A B C      # 将 A B C 归档到一个包里
tar czvf ABC.tar A B C     # 将 A B C 归档到一个包里，并启用压缩

# tar xvf ABC.tar          # 将归档包解压
tar xzvf ABC.tar.gz        # 解压归档压缩包
```